Computers & Security 154 (2025) 104440

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

A novel symbolic model for analyzing Internet of Things protocols based on event logic

Zehuan Li [a,b,*], Meihua Xiao [a,*], Yangping Xu [a], Fangping Chen [c], Huaibin Shao [c], Sufen Yan [d]

[a] School of Information and Software Engineering, East China Jiaotong University, Nanchang, China
[b] School of Electrical and Automation Engineering, East China Jiaotong University, Nanchang, China
[c] Jiangxi University of Software Professional Technology, Nanchang, China
[d] Jiangxi Provincial Party School of CPC, Nanchang, China

ARTICLE INFO

Keywords:
Event logic
Internet of Things protocols
Formal analysis
Symbolic model

ABSTRACT

The unique characteristics of the Internet of Things (IoT) make IoT protocols more vulnerable to security risks compared to traditional security protocols. Formal methods are widely used to analyze and verify the security properties of IoT protocols, but existing methods often fail to address the special security mechanisms required in the complex IoT network environment. A novel symbolic model based on event logic is proposed to implement the comprehensive analysis of secure IoT protocols. The model enhances the description of message flow and match, while also extending the inference capabilities of equivalence and injectivity. Aiming at the pre-shared key, cookie, raw public key, and certificate authority in the IoT protocols, a proof system of the model is proposed to prove the security properties of these mechanisms. To improve efficiency, the match buffer reduction method is also presented to eliminate the redundant proof steps. Formal analysis and verification of protocols, including DTLS, SRTP, CoAP, and PUF, uncover several attacks that violate security properties. The results reveal that the proposed symbolic model is an effective method for the formal analysis and proof of IoT protocols.

Introduction

The popularity of IoT technology has led to the widespread presence of IoT devices across both industrial (Karam et al., 2024; Riyadi et al., 2021) and civil (Zanella et al., 2014; Peng et al., 2024) fields. IoT systems typically follow a three-layer basic architecture (Kakkar et al., 2021) shown in Fig. 1. The perception layer, which is the fundamental layer of the IoT, is responsible for data collection and perception. The network layer handles the transition of data collected by the perception layer to various devices, platforms, and servers, with core tasks such as data transition and network management. The application layer, the highest layer of IoT, interacts directly with users and specific application scenarios.

Different layers of IoT face distinct security challenges. Research on IoT protocol network security mainly focuses on the network layer, while the application and perception layers are often abstracted as the implementation principal part of the network layer. From the perspective of the network layer, IoT characteristics include limited computing power, limited network resources, dynamic, heterogeneous, physical exposure and large-scale systems. Due to above unique characteristics, IoT cannot rely on the same security mechanisms as general computers, making IoT devices more vulnerable (Fremantle and Scott, 2017). The limited computing power and network resources prevent IoT protocols from using strong security mechanisms. Additionally, frequent changes of IoT nodes and network topology may necessitate the introduction of dynamic session identifiers to prevent network access conflicts caused by complex protocol interactions. The vast scale of IoT networks also makes the IoT protocol more prone to unexpected errors. There are multiple protocols used at the same time in the IoT environment, and this heterogeneity leads to inconsistent security requirements for different parts, and IoT networks are more directly facing attacks at the physical layer than traditional networks, which requires some IoT protocols to have physical non-cloning functions.

The security of the IoT network layer mainly relies heavily on the secure IoT protocol to ensure the secure operation of the entire IoT environment from the network protocol. The IoT protocols generally need to satisfy authentication, confidentiality, integrity, and availability (Hofer-Schmitz and Stojanović, 2020). Authentication can be categorized into aliveness, synchronization agreement, non-injective synchronization, and injective synchronization from weak to strong. It

* Corresponding authors.
E-mail addresses: zehuanli@ecjtu.edu.cn (Z. Li), xiaomh@ecjtu.edu.cn (M. Xiao).

https://doi.org/10.1016/j.cose.2025.104440
Received 8 October 2024; Received in revised form 23 February 2025; Accepted 11 March 2025
Available online 13 March 2025
0167-4048/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

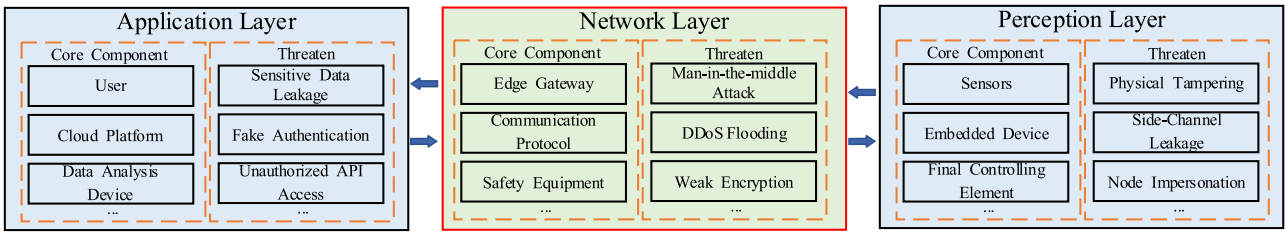

Fig. 1. Basic architecture of IoT.

usually defends against replay attacks, man-in-the-middle attacks, impersonation attacks, and sybil attacks. Confidentiality is one of the most important security properties, a protocol that captures confidentiality can avoid eavesdropping attacks, communication sniffing attacks, plaintext recovery attacks, and privileged insider attacks. The goal of integrity is to avoid data being tampered with without detection, which can avoid modification attacks and black hole attacks. The availability ensures that the protocol format and structure are correct, and can withstand battery exhaustion attacks and Denial of Service (DoS) attacks. In the field of network security, especially in the IoT environment, availability ensures that authorized users can access information and services at any time without interference (Pokorni, 2019; Xiong et al., 2018; Asghari et al., 2018). This means that system and network resources need to be continuously available to meet user requirements. Design flaws in network protocols or unreasonable network architectures can affect the reliability and stability of data transmission, thereby affecting availability. Some protocols may experience congestion or deadlock when handling a large number of connection requests, resulting in service unavailability. In large-scale IoT deployments, the complexity of the network architecture increases the difficulty of management and maintenance, making it prone to availability issues.

The security mechanism of the IoT protocol implements these security properties. For example, the basic security mechanism scheme of ZigBee protocol is implemented by the symmetric encryption algorithm, which ensures the protocol meets the authentication, integrity, and confidentiality without revealing the symmetric key. The LoRaWAN protocol uses the AES encryption algorithm and end-to-end encryption to implement its security scheme, which provides authentication and integrity assurance to protocol-using devices and servers. The security scheme of the 6LoWPAN protocol is similar to ZigBee, using the symmetric encryption algorithm to achieve the authentication and confidentiality of the protocol. Security analysis of IoT protocols essentially involves analyzing the security mechanisms of the protocols. However, due to the limited hardware facilities and network bandwidth of the IoT devices, the security mechanisms are simplified versions of those used in general computing protocols. Solutions such as pre-shared key (PSK) (Li et al., 2014) and raw public key (RPK) (Wouters et al., 2014) are employed to reduce computational overhead. PSK is installed before devices are deployed, saving considerable computational and network resources during key negotiation. This is particularly useful in group communication protocols. However, devices contain a fixed shared key list, requiring careful protection of the list's privacy. RPK is pre-installed by the device manufacturer and serves as a substitute for the public key, simplifying signature calculations. The IoT devices are also often certified through Certificate Authority (CA) to reduce computing in local devices (Luo et al., 2024).

Formal methods are critical for analyzing the security properties of protocols. Common methods include BAN logic, SVO logic, PCL, and other cryptographic protocol logic to deduce and prove the protocol's security. Alternatively, symbolic models are built using specific modeling languages or calculus, followed by verification using automatic tools (Blanchet, 2012; Xiao et al., 2022). While these methods are designed to analyze general security protocols, but the analysis of IoT protocols falls short. BAN logic is an authentication analysis method based on faith, which lacks the analysis of confidentiality, integrity, availability and traceability. SVO logic, a variant of BAN, suffers from similar limitations. PCL supports authentication and confidentiality proof, but there are defects in availability analysis, and its proof system lacks inference rules about message agreement and data form. Existing protocol analysis logic has similar problems, which cannot establish an effective IoT protocol model, lack inference rules in line with IoT protocol characteristics, and do not support the proof of all security properties.

Event logic is a formal method based on structured reasoning and message matching, mainly used for the description and analysis of network protocols in distributed systems (Xiao and Bickford, 2009). Compared with BAN (Yogesh, 2020), SVO (You, Yoshiaki, Kouichi, 2011), PCL (Yu, Zhi-Yao, Ze-Peng 2021), and other protocol logics, event logic has stronger protocol expression ability, more accurate protocol modeling ability, supports multiple cryptographic primitives, and carries out necessary constraints on protocol information data types. The event logic has been successfully applied to the security proof of PUF, RCIA, RFID, and others (Xiao et al., 2023; Zhong et al., 2022; Song et al., 2022). Therefore, this paper proposes a special analysis method for IoT protocol on the basis of event logic. The event logic supports mechanized proof of the protocol in the NuPrl prover (Bickford and Constable, 2010), but this is only a semi-automatic proof method, and the proof process still requires manual intervention. The state-of-the-art automatic proof tools for protocol security include ProVerif (Blanchet, Vincent, Véronique, 2022), Scyther (Yang, Vladimir, Andreas, 2016), Tamarin (Nakabayashi and Yuki, 2021), etc. The event logic uses events as the basic unit of model construction and proof reasoning. The events are very similar to facts in Tamarin, so we choose Tamarin as the formal tool to realize event logic automatic proof. This paper proposes a symbolic model based on event logic for the security analysis of IoT protocols at the network layer. The main contributions are as follows:

1) The definitions and inference rules of event logic are extended to cover the basic IoT architecture, message equivalence, form equivalence, patterns, sharing, injectivity, and verification match. Predicates are introduced to enhance the description of message flow in event logic, and formal definitions for all IoT protocol security properties are provided.
2) A proof system for security mechanisms such as PSK, RPK, CA, and cookies in IoT protocols is proposed, along with soundness proofs.
3) A match buffer reduction method is introduced to improve the proof efficiency of original event logic. This simplification of the proof process enhances the method's usability and facilitates automatic proof.
4) The proposed model is applied to prove the security of IoT protocols. Proof results show that the DTLS, SRTP, PUF protocols violate security properties. For further research, we provide the complete Tamarin model code.[1]

[1] Our implementation is available at https://github.com/podusunny/Verification-Code.git.

The rest of the paper is organized as follows: 'Related Work' introduces current related work. 'Construction of Novel Symbolic Model' proposes a symbolic model based on event logic to realize formal analysis and proof of IoT protocols. 'Case Study' detailed interprets the implementation and automated verification of the proposed model. 'Security and Performance Evaluation

' evaluate security analysis capability and function of the symbolic model. 'Discussion' discusses study results and the practical application of the model in real-world IoT systems. 'Conclusion and Future Work' concludes this paper and discusses possible improvements to the model and future direction.

Related work

The widespread use of IoT protocol has attracted a large number of researchers to research how to guard its security. Vattakunnel et al. (2016) established a compact verification model for full-state space search for CoAP protocol using model checking tool SPIN, however this model is built for the function application of the protocol, and the security mechanism of the protocol is not fully considered. Kim et al. (2017) proposed an automatic protocol analysis method to discuss whether the protocol can meet the authentication, confidentiality, and availability under a variety of threat models, but this scheme does not support integrity analysis. Chen et al. (2022) used CSP to verify the group communication security and focused on data integrity, but the scheme neglected the analysis of other security properties. Wang et al. (2022) proposed a method to automatically evaluate the security of IoT message protocol. While this method shows promise, it is based on fuzzy detection, which lacks detailed formal specifications for the protocol. The literature (Akhtar and Zahoor, 2021) proposed a model for MQTT protocol verification using the PlusCal language to verify protocol safety and aliveness, taking into account broker roles during protocol implementation. Similarly, literature (Hcine and Hafaiedh, 2018) proposed a general modeling method for the MQTT protocol based on temporal automata. This method supports the modeling and analysis of MQTT protocols at different scales and quality-of-service, with automatic verification through the UPPAL model checking tool.

Regarding the security requirements of IoT protocols, literature (Hasan et al., 2021) researched the selection of protocol encryption schemes, primarily considering the impact of different encryption schemes on protocol efficiency and performance. However, their research does not consider that the selection of cryptographic schemes is related to the security properties of protocols. The literature (Liu et al., 2022) further reduced downlink communication and signaling costs by deploying complex computing on the server side and delegating bulk authentication and access control to the gateway. This scheme uses the Tamarin prover for security proof.

The literature (Wang et al., 2021) researched the security of SKKE protocol employing provable security and gave the game-based security proof scheme of ZigBee protocol (ZigBee protocol is the instantiation protocol of SKKE protocol). The literature (Ping et al., 2020) also applied provable security method to assess the special mechanism of the protocol, offering insights into the resilience of shared key scheme protocols. Compared to traditional formal methods, provable security provides stronger security proofs. However, provable security method lacks the support of automated tools, are unable to analyze large-scale protocol systems, and do not provide specific attack paths. Therefore, this paper does not adopt provable security methods for analyzing IoT protocols.

Construction of novel symbolic model

Brief introduction of event logic and its defects

The event logic is a formal method based on theorem proving, which is mainly used for the formal analysis of distributed systems. According to the characteristics of security protocol, the basic event logic includes protocol description language, event system, axiom system, and security property definition. The protocol description language is used to precisely abstract the action of the security protocol, while the event system is the protocol action eventalized transformation rules, the axiom system and the security property definition are a subset of proof systems. The main symbols and symbolic semantics of the event system are shown in Table 1.

Table 1
Basic symbols and semantics.

Symbol	Semantic	Symbol	Semantic
Id	Protocol agent type	*loc(e)*	Location of event *e*
Atom	Atomic set	*Honest(A)*	Honest agent *A*
Data	Data set	*Send(e)*	Send event
Key	Key set	*Rcv(e)*	Reception event
E	Event set	*Verify(e)*	Verification event
m	Message or data	$e_1 \leftarrow^m e_2$	The m flows from e_2 to e_1
Hr(I)	Holds the matching result of the basic sequence *I*	*Sig(e)*	Signature event
e	Event	*MatchingKeys* (k_1,k_2)	k_1 and k_2 are matching keys
K, *k*	Key	*e* || m	The *e* is logic independent of m
A, B	Protocol agent	*e has* m	The *e* owns m
Where (A)	Located in agent *A*	$e_1 < e_2$	Event orderings denote e_2 occurs after e_1
*Firstsend(A,*m*)*		Agent *A* sends message m for the first time.	

As a powerful key exchange protocol analysis method, event logic can accurately describe the protocol interaction model, and provide a protocol-proof system containing axioms, theorems, and inference rules to prove the authentication and confidentiality of the protocol. However, the application of event logic in the process of IoT protocol analysis still has the following defects.

1) The ability of event logic to describe message flows is insufficient. The original event logic supports the analysis of the message flow, where the data flow symbol $\leftarrow^m$ describes the direction of the message, and from the event point of view, the symbols *has* and || can describe the relation between the key, data, atomic message, and the event. However, these symbols cannot be used to analyze data generation in IoT protocols and data flow from a single perspective, which is critical for analyzing multiple security properties of the protocol.
2) The event logic only supports authentication and confidentiality proofs. The event logic lacks analysis ability of strong authentication, integrity, availability, and traceability, but the IoT protocol specification usually stipulates that the protocol needs or can optionally meet these security properties, which limits the application scope of event logic.
3) The proof process of event logic is too redundant. The event logic proves the authentication and confidentiality of the protocol by matching the message and data structure between the basic sequence, which overcomes the problem that PCL cannot establish the previous action in the thread, but because the basic sequence contains the previous event, repeated matching leads to redundant proof steps, which is not conducive to automatic proof.

The extension of event logic

Symbolic model

Given the defects of the above event logic, this section adds relevant definitions and key theorems, introduces the first receiving predicate and the origin predicate to strengthen the description of the message flow in event logic, and introduces the concepts of message equivalence, form equivalence, pattern, and share to deconstruct the composition of

the message, and proposes a definition of injectivity to specify injectivity property of protocol, as well as put forward the verification match to establish the correlation between different agents. In addition, to support the analysis of IoT protocol's limited network resources, dynamics, heterogeneity and other characteristics, we define IoT network environment and protocol characteristics, and propose axioms and theorems for IoT protocol characteristics.

Definition 1. IoT Network Environment. The IoT network environment is defined as a 6-tuple $< N, E, K, M, \sigma, n_0 >$.

1) The N is a set of protocol agents, representing all nodes in the IoT system.
2) The E is an event set, which contains all events used by the protocol agents.
3) The K is a set of keys that contains all the keys generated by the protocol agents.
4) The M is a message set that contains all the interaction information generated by the protocol agents.
5) The σ is transition set of protocol agents
6) The n_0 in initial agent.

Definition 2. Heterogeneity. The heterogeneity means that different protocols and agents have different security requirements. Follow the following rules:

1) The security requirements are security properties that the protocol agent needs to meet.
2) The security requirements of the same agent executing different protocols are determined by the protocol executed.
3) Security requirements are combined security requirements, that is, if the agent can separately ensure the security requirements of different protocols, the overall security requirements are met.

Definition 3. Dynamic Node. The dynamic behavior of the node is regarded as the function switching of the protocol agent, that is, the agent A has the instruction message m to switch the agent function, satisfy $m \neq m' \wedge Origin(A,m) \wedge Origin(B,m') \wedge Send(m) < Send(m')$.

Definition 4. Message Equivalence. The data m, n, if n can be derived directly from m, then call it $m \mid\!\rightarrow n$. The following rules are available:

1) $m \mid\!\rightarrow m$
2) $m \mid\!\rightarrow n_1 \wedge m \mid\!\rightarrow n_2 \mid = m \mid\!\rightarrow (n_1, n_2)$
3) $m \mid\!\rightarrow (n_1, n_2) \mid = m \mid\!\rightarrow n_1 \wedge m \mid\!\rightarrow n_2$
4) $m \mid\!\rightarrow n_1 \wedge m \mid\!\rightarrow n_2 \mid = m \mid\!\rightarrow \{n_1\}_{n_2}$
5) $m \mid\!\rightarrow \{n_1\}_{n_2} \wedge m \mid\!\rightarrow n_2 \mid = m \mid\!\rightarrow n_1$
6) $Origin(A,m) \wedge m \mid\!\rightarrow n \mid = A\ has$ n

Definition 5. Pattern. Given a set $K \in Key$ of keys and a message m, convert m to a pattern by defining the following function P. The following rules are available:

1) $P(k,K) = k$
2) $P(i,K) = k$
3) $P((m,n),K) = (P(m,k), P(n,k))$
4) $P(\{m\}_k, K) = \{ \begin{matrix} \{P(m)\}_k (k \in K) \\ \{\square\}_k \end{matrix}$

Definition 6. Form Equivalence. The messages m and n are form equivalence if and only if $P(m) = P(n)$, denoted as $m \cong n$. The following rules are available:

1) $P(0) = P(0) \mid = 0 \cong 0$
2) $P(0) \neq P(1) \mid = \neg(0 \cong 0)$
3) $P(\{0\}_k) = P(\{1\}_k) \mid = \neg(\{0\}_k \cong \{1\}_k)$
4) $P(\{\{0\}_{k'}, 0\}_k, k) = P(\{\{1\}_{k'}, 0\}_k, k) \mid = (\{\{0\}_{k'}, 0\}_k, k) \cong (\{\{1\}_{k'}, 0\}_k, k)$
5) $P(\{1\}_k, \{0\}_{k'}) = P(\{1\}_k, \{0\}_k) \mid = (\{1\}_k, \{0\}_{k'}) \cong (\{1\}_k, \{0\}_k)$

Definition 7. Share. Let a secret key k be divided into n segments, k^j represents the secret share of the j segment of the key k, where $j \leq n$, s(k) is all the shares in k, and if k is the key set, then s(k) is the shared set of all the keys in the key set k. If $j = 1, 2..., n$, the following rule can be obtained:

1) $k^1 \wedge k^2 \wedge ... \wedge k^n \mid = k^j \mid\!\rightarrow k$
2) $Origin\left(A, k^j\right) \mid = Origin(A,k)$
3) $Origin\left(A, k^j\right) \mid = Origin(B,k)$ if and only if the event $Rcv\left(k^j\right)$ exists in agent B and k^j is explicit.
4) $E\left(k^j\right) \mid = E(k)$
5) $E_1\left(k^1\right) \wedge E_2\left(k^2\right) \wedge ... \wedge E_n(k^n) \mid = k^j \mid\!\rightarrow k$ if and only if events all occur between the same agents and k in event E is explicit.
6) $E_1\left(k^1\right) \wedge E_2\left(k^2\right) \wedge ... \wedge E_n(k^n) \mid \neq k^j \mid\!\rightarrow k$ if and only if the events do not all occur between the same agents, or the k in the event E isn't all explicit.
7) $P\left(k^j\right) \mid = P(k)$
8) $P\left(k^j\right) \cong P(k)$
9) $A \leftarrow^{k^j} B \mid = Ahask$
10) $A \leftarrow^{k^j} B \mid = E(Rcv) < E(Send)$ if and only if honest agent A has a Rcv event and honest agent B has a $Send$ event.

Definition 8. Injectivity. If a session is executed only once, it is called an injective session, and if all sessions in the protocol are executed once, it is called an injective protocol. If the protocol has events $Send(m)$ and $Rcv(m)$ to form an injective session, then there is $Send(m) =_o Rcv(m)$, and if we consider that $Send(m)$ occurs before $Rcv(m)$, then there is $Send(m) <_o Rcv(m)$.

Definition 9. Verification Match. For the verification event *Verify(*m*, m')*, m is the established data and *m'* is the verified data. *Verify(*m*,m')* matches the generation event or function of the parameter *m'*. Such as *Verify(*m*,m')* matches *Sig(m',*d*)*.

Definition 10. First Receiving Predicate. The predicate *Firstrcv(A,* m*)* indicates that the first time protocol agent A has a message m is through the event *Rcv*.

The *Firstrcv* predicates establish the description of the message passive flow between agents, that is, the origin of the received data is considered from the message receiver viewpoint, which is crucial for the formal description of availability, injective synchronization, and integrity.

Definition 11. Origin Predicate. The predicate *Origin*(*A,*m) indicates that the message m is generated from the protocol agent A, that is, in the entire protocol interaction process, m appears for the first time in the event of A.

In essence, predicate *Origin* is the reverse thinking of predicate *Firstrcv, Firstrcv* starts from the event *Rcv* of the target agent to consider the *Send* event of another agent to establish the data flow relation from outside to inside, and *Origin* starts from the agent to consider the data flow relation from inside to outside.

The predicates *Firstrcv* and *Origin* describe the initial relation between the protocol agent and data, which helps to extract the origin characteristics of key, nonce, tag, and so on from the protocol interaction process, that is, to judge the relation between the protocol agent and data generation. The protocol interaction is the flow of specific data between protocol agents, so it is necessary to accurately describe the data flow. We use these two predicates to formalize availability,

integrity, confidentiality, traceability, and strong authentication (injective synchronization).

IoT devices usually operate under limited network resources and network traffic, their hardware cannot support complex cryptogram calculations, so it is necessary to install a PSK list that supports the actual operation of the cryptographic protocol before deployment, which is a compromise between security and transmission efficiency. The pre-installed PSK list generates the initial key of the security protocol, and the protocol agent randomly selects the key and its label from the PSK list. The confidentiality of the key is completely satisfied by the security of the PSK list. Intuitively, the same PSK list can acquire the same key according to the label. This process is called the Offline Public Shared Key (OPSK) model, and its definition and theorem are given.

Definition 12. **OPSK Model.** Assume that the PSK list has been installed for any honest agent, that is, the honest agent A generates PSK k_1, then agent A has an identification *hint* corresponding to k_1, and can obtain k_1 through this identification; otherwise, the agent with k_1 can also obtain the identification *hint*.

Theorem 1. *Assume that protocol P has Honest(A), and its basic sequence has relation Origin(A,k_1) for k_1, then agent A owns k_2, has $k_2 \mid\!\rightarrow k_1$, $k_1 \mid\!\rightarrow k_2$, and Origin(A, k_2).*

Proof. According to the definition of the OPSK model, the PSK has its corresponding identity, and the disclosure identity is equivalent to the disclosure of the PSK, so k_2 is derived from honest agent A, and k_2 can be derived from each other between k_1.

Lemma 1. ***PSK Release.*** *Assume that any honest agent A contains the generation of PSK k, then there must be no event Send to release k in agent A.*

Proof. According to the honesty axiom, the honest agent does not have an event *Send* that explicitly contains any key, that is, the key is not released to the attacker.

Lemma 2. ***Equivalence.*** *If honest agent A is an agent in a single protocol session, and agent A contains PSK k_1, then the protocol session must contain an honest agent B other than A, and honest agent B generates a key k_2, and k_1 is essentially equivalent to k_2.*

Proof. According to the definition of the OPSK model, agent A executing protocol P generates PSK k_1 and identity *hint*. Assume that agent B executing protocol P does not have PSK and identity equivalent to that of A. Agent B interacts with agent A. When B receives the data constructed by A with k_1, A does not release the PSK and identity according to Lemma 1, then B can analyze the data constructed by k_1, B must contain the key equivalent to k_1. Hence the contradiction.

The IoT protocol adds a cookie mechanism as a component to defend against DoS attacks, which is defined below.

Definition 13. **Cookie Model.** The agents A and B, as the initiator and responder of protocol P respectively, perform protocol operations, and then a cookie can be obtained through the intersection of the basic sequence of A and the basic sequence of B, if and only if there is a cookie mechanism in protocol P.

The RPK is an alternative or supplement to the public key in a limited network environment, and there is a strong relation between the RPK and the public key. When any agent obtains the RPK, which also obtains the corresponding public key, and vice versa. The detailed definition is as follows:

Definition 14. **RPK Model.** If agent A has the RPK k, and there is *Origin*(A,k), then it follows that agent A has the key k' and *Origin*(A,k'). Furthermore, k deduces k', and k' can restore k.

Theorem 2. *The RPK matches the public key.*

Proof. According to the definition of the RPK model, under the RPK model, the public key must have a unique RPK, so that the two sides can deduce each other. The lemma can be proved.

Theorem 3. *Suppose that agent A starts a session with agent B by instructing* m *and agent B is always online, then there is message m', which contains rule* $Origin(B,m') \wedge Send(m') > Rcv(m)$.

Proof. Agent A generates instruction m to obtain $Origin(A,m)$, while the session started by agent A with agent B can be regarded as the dynamic behavior of agent A and agent B. Then $Origin(B,m') \wedge Send(m) < Send(m')$ can be obtained according to the definition of dynamic node, and m' is the dynamic response message of agent B. Then the response of agent B to instruction m must occur after the $Rcv(m)$ event, so $Rcv(m) < Send(m')$ is obtained.

The third-party certificate issuers are considered to be absolutely secure in protocol implementation, which follows the honesty axiom, and their signatures follow the disjoint axiom.

Definition 15. **Honest CA Model.** If the third-party certificate issuer in protocol P is an agent CA, then the CA is honest in the protocol, and the event *Send* in its basic sequence does not explicitly contain the private key corresponding to the signature. Moreover, no other client has a key paired with a certificate verification public key.

The above extension gives event logic a stronger ability to abstract IoT protocols, so we propose new axioms for the improved proof system of event logic.

MORI $New(Na) \wedge E(New) \subseteq I^A \Rightarrow Origin(A,Na)$

ORIF $\neg Origin(A,m) \wedge Ahasm \Rightarrow Firstrcv(A,m) \vee Firstrcv(A,\{m\})$

PAT $P(m) = P(m') \wedge m \mid\!\rightarrow m' \Rightarrow m = m'$

FLOW $I^A \rightarrow^m R^B \wedge Origin(A,m) \Rightarrow Firstrcv(B,m)$

INJ $Rcv(A,m) \wedge Send(B,m) \wedge m \mid\!\rightarrow n \wedge New(n) \Rightarrow Rcv(A,m) =_o Send(B,m)$

INJS $Rcv(A,m) =_o Send(B,m) \wedge Send(B,m) < Rcv(A,m) \Rightarrow Send(B,m) <_o Rcv(A,m)$

OWF $HMAC(n,m) \vee PRF(n,m) \Rightarrow \neg(n \mid\!\rightarrow m)$

VERO $Verify(m,m') \Rightarrow Send(n) \wedge Rcv(n) \wedge n \subseteq m'$

VERM $Verify(m,m') \subseteq I^A \Rightarrow \exists e : E.e \subseteq I^B \wedge MatchV(Verify,e) \wedge A \neq B$

TOPO $I^A \wedge Firstrcv(B,m) \wedge Firstrcv(A,m') \wedge Send(m) < Send(m') \Rightarrow Origin(A,m)$

HETE $Send(m) \subseteq I^A \wedge Send(m') \subseteq R^A \wedge m = m' \Rightarrow \mathrm{Rcv}(m) \subseteq R^B \neq Rcv(m') \subseteq I^B$

ESID $Origin(A,m) \wedge Rcv(m) \Rightarrow Firstsend(A,m)$

Extended axioms include message origin axiom, origin message flow axiom, pattern axiom, flow axiom, injectivity axiom, injective synchronization axiom, one-way function axiom, verification origin axiom, verification match axiom, topology axiom, heterogeneity axiom, ephemeral session identity axiom. The message origin (**MORI**) axiom indicates where the message (nonce or key) was generated. The origin message flow (**ORIF**) axiom states that agent A owns data m but does not own the generation event of m, so for A, m must be derived from the event *Rcv*. The pattern (**PAT**) axiom states that if both message equivalence and pattern equivalence are satisfied, then the data is equal. The flow (**FLOW**) axiom indicates the direction of the data in the basic sequence. The injectivity (**INJ**) axiom states that if a message session has a recognizable nonce, then the session is unique. The injective synchronization (**INJS**) axiom states that there is injective synchronization between sessions if there is an order relation $E(Send) < E(Rcv)$ between the *Send* events and the *Rcv* events on the basis of the injective sessions. The one-way function (**OWF**) axiom states that for the result n in a one-way function event, n cannot derive other safe parameters m in a one-way function event, expressed as $\neg(n \mid\!\rightarrow m)$. The verification origin (**VERO**) axiom states that if the agent has an event $Verify(m,m')$, then

there must be a session, and the data that needs to be verified is contained in the message flow of the session. The verification match (**VERM**) axiom states that for a basic sequence containing an event *Verify*, there must be a different basic sequence containing an event matching the *Verify*, which generates the verified data. The topology (**TOPO**) axiom indicates that the change of the object behavior of the agent causes the change of the specific interaction behavior, which is reflected in the change of the event *Send*. The heterogeneity (**HETE**) axiom indicates that in situations where entities play different roles, events cannot be instantiated to the same event. The ephemeral session identity (**ESID**) axiom indicates that the session ID is propagated outward through the event *Rcv*.

The availability of a proof system depends on whether the system has soundness. The soundness of event logic comes from the fact that it contains a series of axioms, theorems, lemmas, and rules that are valid and correct in proving the formula constructed by event logic. We give the following soundness proof of extended event logic.

Theorem 4. *Assume* Γ *is the set of all axioms, theorems, lemmas, and rules in event logic, and* Y *is the set of basic formulas in event logic. The extended event logic is soundness if and only if* $\Gamma \mid\rightarrow \mathrm{Y} \Rightarrow \Gamma \mid = \mathrm{Y}$.

Proof. The injectivity axiom and injective synchronization axiom are taken as proof cases. For events $Rcv(A,m)$ and $Send(B,m)$, there must be a match session between protocol agents A and B. According to the definition of injectivity, if the session can only be executed once, then the session can meet the injectivity property. The event *New(n)* indicates that n is unique because *New* does not produce two identical data in any case. In this way, when m can directly derive n, exactly as $m \mid\rightarrow n$, it means that the session is marked by a unique n. Therefore, $Rcv(A,m) =_o Send(B,m)$ is true. When the events $Rcv(A,m)$ and $Send(B,m)$ have order, on the basis of injectivity axiom proof, according to the injectivity definition, $Send(A,m) <_o Rcv(B,m)$ can be obtained. The soundness proof of other axioms uses various existing or extended definitions, rules, theorems, and lemmas of event logic. The proof process is similar to the proof of injectivity axiom and injective synchronization axiom, which will not be repeated.

Security properties

The security properties are the specification requirement that the protocol needs to meet, and also the result that the system needs to prove. Secure IoT protocols usually need to meet the requirements of authentication, integrity, confidentiality availability, and traceability. The original event logic is mainly a protocol analysis tool for authentication and confidentiality . In order to enable event logic to support the analysis of secure IoT protocols, we propose the definition of availability and integrity by event logic, a definition of injective synchronous based on the original authentication definition, and a more rigorous definition of confidentiality by introducing the adversary model.

Definition 16. **Availability.** If an honest agent A has data derived from the receiving event *Rcv*(m), then there must be an honest agent B has event *Send*(*m*'), *Send*(*m*')<*Rcv*(m), and $m \cong m'$.

Definition 17. **Integrity.** The honest agent A obtains data m through event *Rcv*; honest agent B has event *Send*(*m*') and relation *Origin*(*B*,*m*'). If $P(m) = P(m') \wedge m = m'$ can be obtained, then the data m between A and B capture integrity.

Definition 18. **Confidentiality.** Confidentiality is protection of specific secrets. It means that the honest agent's session keys k will not be released by event *Send* and cannot be obtained by the adversary, and confidential messages flow strictly according to the specified policy to ensure that only the specified events *Send*(m) and *Rcv*(m) exist and meet the $Send(m) \rightarrow^m Rcv(m)$.

Definition 19. **Traceability.** Traceability is viewed as a supporting security mechanism or security capability. Traceability means being able to track where data comes from and where it goes. That is, agent A has data d, and d comes from agent B, which has relation $Origin(B,d)$, and there is a channel between A and B, which has $Send(d) \rightarrow^d Rcv(d)$.

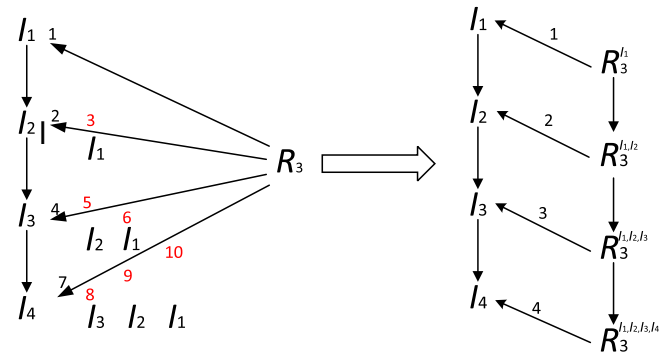

Fig. 2. The original method to the MB reduction method.The left is the original event logic proof step, the right is the improved step, the numbers are the number of matches, and the numbers marked in red are the number of redundant proofs.

Definition 20. **Injective Synchronization (I-SYNCH).** The injective synchronization is strong authentication. For agent A's receiving event *Rcv*(m), the existence of a unique honest agent B contains the sending event *Send*(*m*'), $Send(m') <_o Rcv(m)$, and $m = m'$.

Analysis and proof process

The event logic divides the protocol fragments into basic sequences, and then it analyzes the relation between the basic sequences of different agents according to the order of matched parties from front to back. Compared with PCL, this analysis method can take into account the previous actions of the agent, but there will be redundant proof in this process, and it will increase with the increase of the size of the analyzed protocol. Therefore, we propose the Match Buffer (MB) reduction method to solve the problem of proof redundancy. We give some definitions of protocol and basic sequence.

Definition 21. **Role and Agent.** The role is a party that runs a protocol in a protocol specification, and an agent is a member that plays a protocol role. Multiple agents can play the same role.

Definition 22. **Action and Event.** The action refers to the operation that a protocol role needs to perform according to the protocol specification. The event is an abstraction of a protocol action, indicating the behavior of a role agent in a protocol.

Definition 23. **Protocol Model and Instantiation.** The protocol model is a set of events of agents that play the roles of different protocols, which represents the basic interaction process of protocols. Instantiation, which assumes the actual operation of the protocol, will give specific functional meaning to the agent event.

Definition 24. **Basic Sequence and Its Match.** The basic sequence is a series of sequentially arranged sets of events produced by the protocol agent following the protocol operation, identified as I^*, R^*, *=1, 2, 3, …, n, which respectively represent the initiator basic sequence and responder basic sequence. If there are events in basic sequence I that match the event of R, for example, event *Send* matches event *Rcv*, and other events in I are instantiated according to messages of match event, and the I of the instantiation conforms to message equivalence, then the basic sequence R matches basic sequence I. The symbol is $MatchBS(I,R)$.

The original event logic advances the proof process by a progressive match method. As shown on the left in Fig. 2, assume that the initiator of

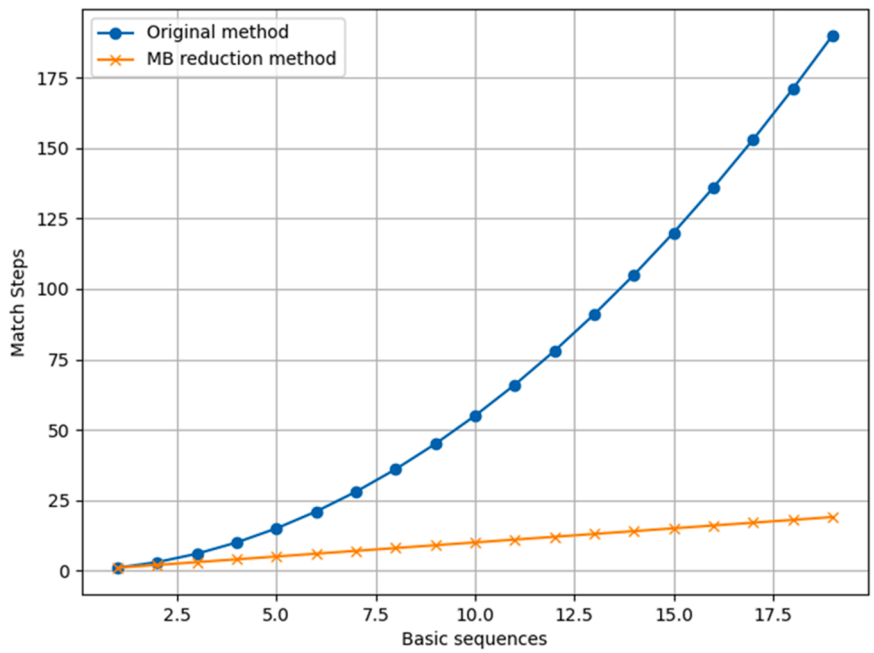

Fig. 3. Proof efficiency comparison.

protocol P is divided into the basic sequence I_1, I_2, I_3, and I_4; the responder is divided into R_1, R_2, and R_3. The corresponding relation between the basic sequences is that the sequence with the larger label contains the sequence with the smaller label, that is $I_i \supseteq I_j$ if and only if $i \geq j$. When considering the entire interaction process of the protocol, the analysis starts from the perspective of basic sequence R_3. Firstly, the match relation between R_3 and I_1 is analyzed, if not match, the match relation between R_3 and I_2 is analyzed. At this time, since I_2 contains I_1, R_3 needs to repeat the match process with I_1. A redundant step is generated during the analysis of R_3, I_1, and I_2. When the analysis of R_3 matches I_4, the redundant step is increased to 6. By summarizing the above process, the redundancy growth can be simply obtained as $n(n+1)/2$, n denote numbers of I.

For the problem of original event logic proof redundancy, the MB reduction method can effectively improve proof efficiency and avoid redundancy, as shown on the right in Fig. 2. The MB reduction method performs the following proof steps:

1) The basic sequence R_3 of the initiator matches the basic sequence I_1 of the responder, and the match result is an observable instantiated event formula.
2) The match result is appended to R_3 as a buffer term, denoted as $R_3^{I_1}$.
3) The R_3 makes a second match. The I_2 intersects with I_1, and in the process of matching I_2 with R_3, the intersection between I_2 and I_1 is replaced by the instantiation event of I_3.
4) The R_3 buffers the instantiation event of I_2, denoted $R_3^{I_1,I_2}$.
5) Repeat the above steps to get the final match result.

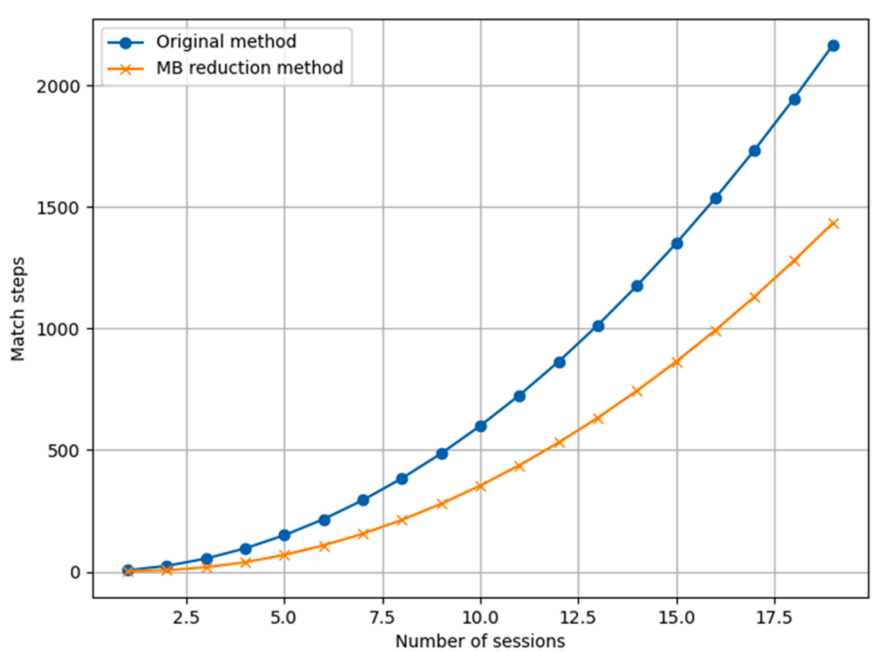

Fig. 4. Scalability comparison under CR.

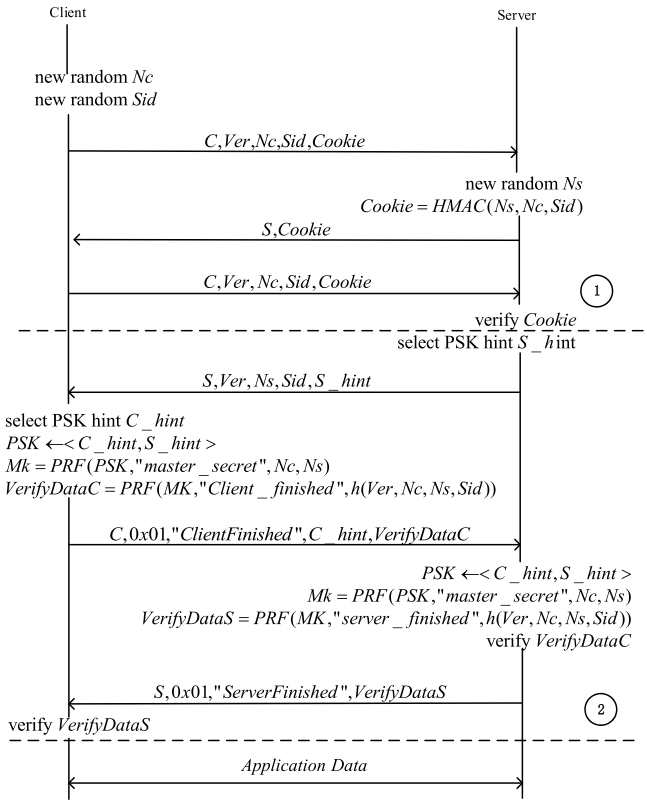

Fig. 5. DTLS protocol interaction process.

6) If there are multiple sessions, the match session is computed first, and the match of subsequent sessions is not involved.

The MB reduction avoids repeated match by replacing the instantiation event of the current sequence. Fig. 2 shows that for the proof process consisting of 4 initiator basic sequences and 3 responder basic sequences, the progressive reduction method can reduce 6 match steps. The MB reduction method reduces the increase in the number of proof steps from quadratic to linear, as shown in Fig. 3, which shows that the improved method is more efficient to prove.

For the overall proof process of security protocol, Appendix A shows the basic idea of proof. Moreover, to better illustrate the scalability of MB reduction method, we use Challenge-Response (CR) protocol as an example to demonstrate this method. The interaction of the CR protocol is as follows:

$$A \rightarrow B : g^a$$
$$B \rightarrow A : g^b, Sig_B\{|g^b, g^a, A|\}$$
$$A \rightarrow B : Sig_A\{|g^b, g^a, B|\}$$

To get eventualization of CR protocol, the basic sequence is as follows:

$$I_1 = New(ga), Send(A, ga)$$
$$I_2 = I_1, Rcv(n, s)$$
$$I_3 = I_2, Verify(n, ga, A, B, s), Sig(s', n, ga, B, A), send(s')$$
$$R_1 = Rcv(A, m), New(gb), Sig(gb, m, A, B, s), Send(gb, s)$$
$$R_2 = R_1, Rcv(s')$$
$$R_3 = R_2, Verify(gb, m, B, A, s')$$

The basic sequence of CR protocol consists of I_1-I_3 and R_1-R_3, so 6 matches need to be calculated using the initial method, and only 3 matches need to be calculated using the MB reduction method. When there are multiple CR sessions, according to proof step 6), the number of matches for the original method increases by $6n^2$ with the increase in the number of sessions, while the number of matches for the MB reduction method increases by $3(1.5n^2 - 3.5n + 3)$ with the increase in the number of sessions. The scalability of MB reduction method is better than that of the original method, as shown in Fig. 4.

Case study

Formal analysis of IoT protocol

We use the symbolic model proposed above to formally analyze and verify DTLS, SRTP, CoAP and PUF protocols. In order to give a concise and clear application process of the novel symbolic model, we introduce the analysis and proof process of DTLS protocol in detail, as well as the automatic verification process. The security proof process of other protocols is given in Appendix B. Through the security analysis and proof of various IoT protocols, it is shown that the symbolic model proposed by us can be applied in the IoT field.

The Datagram Transport Layer Security (DTLS) protocol (Rescorla E and Modadugu N, 2012) is a security assurance protocol designed for IoT environment. It is a security protocol proposed based on TLS protocol that considering the special security mechanism. The DTLS protocol supports PSK, RPK, and CA mechanisms, which can be flexibly applied to IoT systems with different security objectives. This part mainly focuses on the DTLS protocol under PSK mechanism. The analysis and proof processes under RPK and CA mechanism are similar, and the results are directly presented in the automatic verification section.

The DTLS protocol detailed interaction process is shown in Fig. 5.. This process uses the PSK mechanism, so for the following modeling and proof, we use the OPSK model. The implementation of the DTLS protocol can be divided into two stages. In the first stage, the agent server carries cookie verification out, which ensures the realization of DoS resistance. The second stage is to realize key negotiations between the two parties. In this process, the *PSK* and *Mk* are generated as well as *VerifyDataC* and *VerifyDataS* are verified. Correctly verifying *VerifyDataC* and *VerifyDataS* can ensure the authentication and integrity of the protocol. The protocol confidentiality depends on whether the *PSK* and *Mk* can be obtained by the attacker.

According to the interaction process in Fig. 5., the eventalization of the DTLS protocol and the segmentation of the basic sequence can be obtained as follow:

$$I_1 = New(Nc), New(Sid), Send(C, Nc, Sid)$$
$$I_2 = I_1, Rcv(S, cookie')$$
$$I_3 = I_2, Send(C, Nc, Sid, cookie')$$
$$I_4 = I_3, Rcv(S, Ns', Sid, Sh')$$
$$I_5 = I_4, New(Ch), List(Psk, Ch, Sh'),$$
$$PRF(Mk, Psk, Nc, Ns'),$$
$$PRF(Vc, Mk, h(Nc, Ns', Sid), Send(C, Ch, Vc)$$
$$I_6 = I_5, Rcv(S, Vs'), Verify(Vd, Vs')$$

$$R_1 = Rcv(C', Nc', Sid'), New(Ns),$$
$$HMAC(cookie, Ns, Nc', Sid')$$
$$R_2 = R_1, Send(S, cookie)$$
$$R_3 = R_2, Rcv(C', Nc', Sid', cookie'),$$
$$Verify(cookie', cookie), New(Sh)$$
$$R_4 = R_3, Send(S, Ns, Sid', Sh)$$
$$R_5 = R_4, Rcv(C, Ch', Vc'), List(Psk, Ch', Sh),$$
$$PRF(Psk, Nc', Ns),$$
$$PRF(Vs, Mk, h(Nc', Ns, Sid)),$$
$$Verify(Vs, Vc')$$
$$R_6 = R_5, Send(S, Vs)$$

Proof process

According to Definition 20, if the DTLS protocol satisfies injective synchronization, the formula must be satisfied:

$$\bigcap_{i=1}^{n} MatchBS(R_i, I_i) \wedge E_i(Rcv) =_o E_i(Send)$$

Further simplification gives the formula:

Table 2
Injective synchronization proof of DTLS protocol.

Formula	Tag	Explanation
1) $R_6 \Rightarrow \exists e_1 : E.e_1 = Send(S, Vs)$	R_6	Eventalized basic sequence R_6
2) $e_1 \Rightarrow \exists e'_1 : E.e'_1 = Rcv(S, Vs)$	1), AxiomR	According to AxiomR, there must be *Rcv* events that match e_1.
3) $I_1 \wedge e'_1 \Rightarrow \exists e''_1 : E.e''_1 \subseteq I_1 \wedge \neg(P(e''_1) = P(e'_1)) \Rightarrow \neg MatchBS(R_6, I_1)$	1), 2), PAT	The step 3) start to consider the events in the basic sequence of the initiator, I_1 contains event $Send(C, Nc, Sid)$, which cannot be form equivalence with e_1, so combining 1), 2) and axiom PAT can get R_6 and I_1 do not match. The use of the axiom PAT focuses on the equivalence of the pattern of events and messages in the basic sequence, and the inequivalence between the $Send(C, Nc, Sid)$ and $Rcv(S, Vs)$ patterns in step 3 obviously does not satisfy the form equivalence.
4) $I_2/I_1 \wedge Hr(I_1) \Rightarrow I'_2$	–	The matching results of the basic sequence I_1 are held, and the I_1 part of the basic sequence I_2 is excluded to obtain the partial sequence I'_2.
5) $I'_2 \wedge e'_1 \Rightarrow \exists e''_1 : E.e''_1 \subseteq I'_2 \wedge \neg(P(e''_1) = P(e'_1)) \Rightarrow \neg MatchBS(R_6, I_2)$	1), 2), 4), PAT	Similar to 3) proof process, can get R_6 and I_2 do not match.
6) $\neg MatchBS(R_6, I_3) \wedge \neg MatchBS(R_6, I_4) \wedge \neg MatchBS(R_6, I_5)$	The same goes for I_3,I_4,I_5	Similarly, I_3, I_4 and I_5 in the basic sequence of the initiator do not match R_6
7) $I'_6 \wedge e'_1 \Rightarrow \exists e''_1 : E.e''_1 \subseteq I'_6 \wedge P(e''_1) = P(e'_1) \wedge E(e''_1) =< S, Vs > \Rightarrow MatchBS(R_6, I_6)$	1), 2), PAT, AxiomR	The I'_6 contains event *Rcv*, and it has the form and message equivalence as event e_1, so R_6 matches I_6
8) $MatchBS(R_6, I_6) \wedge Verify(Vd, Vs') \subset I'_6 \Rightarrow Vd = Vs'$	7), VERO, VERM	The R_6 matches the I_6 to get the presence message flow. The R_6 contains the event *Verify*, so according to the axioms VERO, VERM, get $Vd = Vs'$. Axiom VERO, VERM focuses on the fundamental properties of *Verify* event, which, along with the process of message delivery, sequence match, can prove the messages equivalence.
9) $Origin(C, Vs) \wedge Rcv(S, Vs') \wedge Send(S, Vs) \Rightarrow Rcv(S, Vs') =_o Send(S, Vs)$	1), FLOW, INJ	On the basis of 1), axiom FLOW can show that S is the first to receive Vs', and then the injective relation between $Rcv(S, Vs')$ and $Send(S, Vs)$ can be obtained according to axiom INJ. The application of the FLOW axiom is in the generation of messages in the basic sequence, where there are new messages there are conditions for using FLOW. The INJ axiom applies to nonce generation and passing, i.e., *Vs* and *Vs'* can be traced back to event *New* in the basic sequence.
10) $MatchBS(R_3, I_3) \wedge \ldots \wedge MatchBS(R_5, I_5)$	1)−7)	Repeat steps 1)−7) to get R_3 matching I_3, R_4 matching I_4, and R_5 matching I_5
11) $Rcv(S, Vs') =_o Send(S, Vs) \wedge Rcv(C, Ch', Vc') =_o Send(C, Ch, Vc) \wedge Rcv(S, Ns', Sid, Sh') =_o Send(S, Ns, Sid', Sh)$	9)	Repeat the steps of 9) to obtain all injective relations in the basic sequence.
12) $R_2 \wedge \Rightarrow \exists e_5 : E.e_5 = Send(S, cookie)$	R_2	Eventalized basic sequence R_2.
13) $e_5 \Rightarrow \exists e'_5 : E.e'_5 = Rcv(S, cookie)$	12), AxiomR	According to AxiomR, there is an *Rcv* event that matches e_5.
14) $\forall e : Rcv.e \neg \subset I_1 \Rightarrow e'_5 \neg \subset I_1$	I_1	Get e'_5 is not in the basic sequence I_1.
15) $I'_2 \wedge e'_5 \Rightarrow \exists e''_5 : E.e''_5 \subseteq I'_2 \wedge P(e''_5) = P(e'_5)$ 15) $\Rightarrow \bigcap_{i=1}^{n} Rcv(m'_i) < Send(m_i) \not\Rightarrow E(e''_5) =< S, Vs > \not\Rightarrow MatchBS(R_2, I_2)$	12)−14), PAT , FLOW, MORI	Combining steps 12)−14), FLOW and MORI indicate the direction of the message, that is, the partial order relation $Rcv(m'_i) < Send(m_i)$. The PAT can conclude that events e'_5 and e''_5 are not message equivalence, so R_2 and I_2 do not match. The axiom MORI characterizes *New* and can be used when *New* occurs in the basic sequence.
16) $P \mid \neq \cap_{i=1}^{n} Rcv(m'_i) <_o Send(m_i) \wedge m_i = m'_i$	7), 10), 11), 15), INJS	According to 7), 10), 11), 15) the proof process can be seen that 16) does not satisfy the axiom INJS, so the protocol does not satisfy the injective synchronization. The INJS axiom is applied when *Send* and *Rcv* have partial order relations and the session is injective.

$$\bigcap_{i=1}^{n} Rcv_i(m'_i) <_o Send_i(m_i) \wedge m_i = m'_i$$

The proof process is shown in Table 2.

The formula below 'Formula' session in Table 2 is the formula to be proved, and the previous steps and axioms needed to prove the formula are under 'Tag' session. The content below 'Explanation' session explains the role of each step, and emphasizes the position and method of using each axiom proposed above when it is used for the first time. The results show that the DTLS protocol does not meet the injective synchronization, mainly because the I_2 sequence of the protocol lacks the operation of cookie verification. The (1)-(7) prove that the basic sequence R_6 matches I_6, and step (10) repeats the process to get a match from R_3 to R_5. The (8) and (9) infer the injectivity of the session through the event *Verify*. The (8) proves the equivalence of message and form in event *Verify*, the (9) proves the existence of injective sessions between agent C and S according to the FLOW axiom and INJ axiom, the (11) proves the existence of injective protocol sessions based on (9). The (12)-(15) indicate that the equivalence of DTLS cannot be deduced. According to (7), (9), (10), and (15), (16) prove that the DTLS protocol does not meet the injective synchronization.

Automatic verification

Tamarin prover

Tamarin prover is one of the state-of-the-art security protocol formal verification tools, which has been successfully used in the security verification of TLS1.3 (Cremers et al., 2017), 5G-AKA (Basin et al., 2018), and others(Li et al., 2020; Tran et al., 2023; Modiri et al., 2023). Tamarin uses a protocol description language based on multiset rewriting rules to build a detailed protocol symbol model, and a constraint-solving algorithm to verify the protocol symbol model, which returns a correct proof result or a counterexample after the verification is terminated. Tamarin supports the Diffie-Hellman exponentiation, bilinear pairings, and equational theory. Security properties in Tamarin are modeled as trace properties, so security properties verification is done either by detecting the trace of the transition system or by relying on the observational equivalence of the two transition systems. Tamarin protocol description language rules based on multiset rewriting include premises (left-hand side), action facts (middle), and conclusions (right-hand side). It constitutes a transition system from one fact set to another. Take the following rule for example: rule example:

[Fr(~ltk)]–>[!Ltk($A, ~ltk), !Pk($A, pk(~ltk))]

This example denotes built-in fate *Fr* of Tamarin prover generates a fresh name *ltk*, then the premises are removed, and get fats *Ltk, Pk*, which marked symbol ! in front means persistent existence. Security properties in Tamarin prover are represented by lemmas, which are described using first-order logic, as an example of the following lemma. lemma message_authentication:

"All b m #j. Authentic(b,m) @*j* ==> Ex #i. Send(b,m) @i &*i* < *j*"

This lemma example denotes authentication, which means that action fate *Authentic*(*b*,m) happens at timepoint *j*, then exists action *Send*(*b*, m) at timepoint variables *i*, and *j* later than *i*.

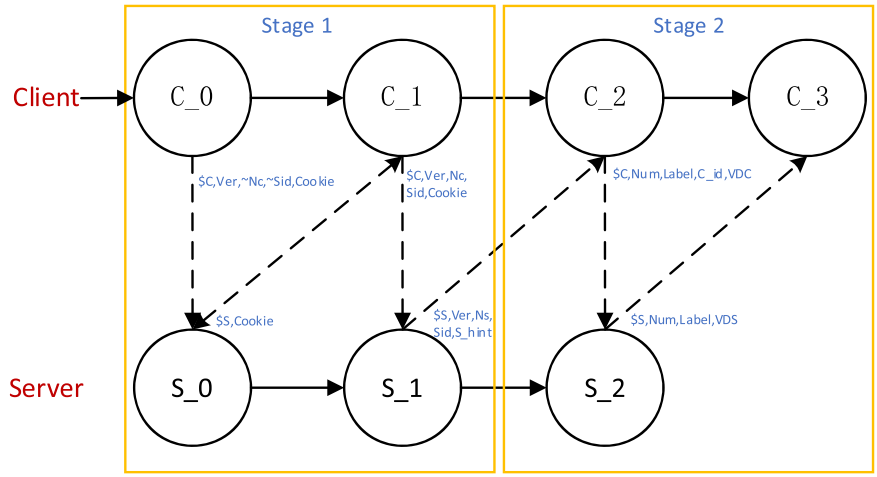

Fig. 6. The overall process of DTLS model in Tamarin.

4.3.2. Assumption

We use the DY model (Cervesato, 2010) as the threat model. The DY threat model can control the entire network, and intercept all messages sent to the network, which can be casually combined and replayed. According to the security mechanism defined in the protocol specification and DY model, we give the security assumption:

Assume that the protocol agents are honest: we believe that the client, server, and CA all strictly follow the protocol execution process, and there is no extra-specified execution manner.

Assume that the cryptographic algorithm is perfect: hash message authentication code, pseudorandom function, hash algorithm, signature algorithm, encryption algorithm, decryption algorithm, and nonce generation are perfect.

Assume that the PSK list is reliable: the adversary cannot know the pre-installed key; the pre-installed key is secure.

Assume that there are no malware plugins: the adversary can use all the cryptographic primitives, but the adversary cannot control the protocol agent action.

Modeling in Tamarin

This session uses the Tamarin prover to realize the automatic proof of DTLS by event logic. The event logic model of the DTLS protocol is aligned with the Tamarin model, and the proof results are consistent with those presented in Table 2. We map the event logic model to the Tamarin model to enable automated verification. For a more comprehensive analysis of DTLS, we have verified the protocol under the PSK, RPK, and CA mechanisms. Since the verification process is similar for all mechanisms, and to maintain consistency with the manual proof process, we focus on detailing the verification process for the DTLS protocol based on the PSK mechanism, along with the corresponding code and comments.

The Tamarin model consists of seven rules. The overall structure is shown in Fig. 6. Rules *C_0, C_1, S_0, S_1* constitute the first stage of the model, and rules *C_2, C_3, S_2* constitute the second stage of the model. *C_** is the client, *C_0* is the initial rule to start running the entire model, and *S_** is the server, which starts running after *C_0* is triggered. Solid lines represent rule state transitions, and dashed lines represent data interactions between *C_** and *S_**. Without considering the attacker, the basic logic of operation is that the client is instantiated to rule *C_0* to start the first stage of the model, *C_0* sends a message to *S_0* and transits to rule *C_1, S_0* receives the message and sends data to *C_1* and migrates to rule *S_1*. After receiving the message from *S_0, C_1* sends the message to *S_1* and transits to *C_2*. After receiving the message, *S_1* transits to *C_2* and then transits to *S_2*. The model enters the second stage. In the second stage, rule *C_2* sends a message to *S_2* and transits to *C_3*, which waits for the final message sent by *S_2*. Combined with the protocol interaction process in Fig. 5, the two stages of the Tamarin model correspond to the two stages of the protocol interaction.

```
1. rule C_0:
2. let Cookie='0' Ver='DTLS1_2' in
3. [Fr(~Nc),Fr(~Sid)]
4. --[CookieG0(~Sid,Cookie),Honest($C)]->
5. [Out(<$C,Ver,~Nc,Sid,Cookie>),ST_C_0($C,Ver,~Nc,Sid,Cookie)]
6. rule S_0:
7. let S='Server' Cookie=HMAC(~Ns,Nc,Sid) in
8. [In(<$C,Ver,Nc,Sid,Cookie>),Fr(~Ns)]
9. --[CookieG1(Sid,Cookie),Honest($S)]->
10. [Out(<$S,Cookie>),ST_S_0($S,Ver,Nc,~Ns,Sid,~Ns,Cookie)]
11. rule S_1:
12. let S_hint='S_hint' in
13. [In(<$C,Ver,Nc,Sid,Cookie>),ST_S_0($S,Ver,Nc,Ns,Sid,Ns,Cookie)]
14. --[Eq(VH(Cookie,HMAC(Ns,Nc,Sid)),true),CookieV(Sid,Cookie),Honest($S)]->
15. [Out(<$S,Ver,Ns,Sid,S_hint>),ST_S_1($S,Ver,Nc,Ns,Sid,S_hint)]
16. rule C_1:
17. [In(<$S,Cookie>),ST_C_0($C,Ver,Nc,Sid,Cookie)]
18. --[CookieG2(Sid,Cookie),Honest($C)]->
19. [Out(<$C,Ver,Nc,Sid,Cookie>),ST_C_1($C,Ver,Nc,Sid,Cookie)]
```

Fig. 7. Stage 1 of DTLS.

```
1. rule C_2:
2. let Label='Finished' Num='0x01' C_id='C_id' PSK=LIST(S_hint,C_id)
3. MK=PRF(PSK, 'MS',Nc,Ns) VDC=PRF(MK,lable,h(Ver,Nc,Ns,Sid)) in
4. [In($S,Ver,Ns,Sid,S_hint),ST_C_1($C,Ver,Nc,Sid,Cookie)]
5. --[MasterKey($C,$S,<Label,PSK,MK,VDC>,Negotiate($C,$S,<Label,PSK,MK,VDC>),
6. Origin(MK),Sd($C,<Num,Label,C_identity,Verify_Data_C>)]->
7. [Out(<$C,Num,Label,C_id,VDC>),ST_C_2($C,Ver,Nc,Ns,Sid,Cookie,PSK,MK,VDC)]
8. rule S_2:
9. let Label='Finished' Num='0x01' C_id='C_id' PSK=LIST(S_hint,C_id)
10. MK=PRF(PSK, 'MS',Nc,Ns) VDC=PRF(MK,lable,h(Ver,Nc,Ns,Sid)) in
11. [In(<$C,Num,Label,C_id,VDC>),ST_S_1($S,Ver,Nc,Ns,Sid,S_hint)]
12. --[MasterKey($S,$C,MK),VerifyC(VDC),Negotiate($C,$S,<Label,PSK,MK,VDS>),
13. Commit($C,$S,MK,PSK),Origin(MK),Sd($S,<Num,Label,Verify_Data_S>),
14. Trace($C,<Num,Label,C_identity,Verify_Data_C>),]->
15. [Out(<$S,Num,Label,VDS>),ST_C_2($C,Ver,Nc,Ns,Sid,Cookie,PSK,MK,VDC)]
16. rule C_3:
17. let [In(<$S,Num,Label,VDS>),ST_C_2($C,Ver,Nc,Ns,Sid,Cookie,PSK,MK,VDC)]
18. --[ VP(VDC,VDS), SessionKey($S,$C,MK), VerifyS(VDS),
19. Commit($C,$S,MK,PSK), Trace($S,<Num,Label,Verify_Data_S>)]->[]
```

Fig. 8. Stage 2 of DTLS.

1.lemma **Dos_protection:**
2."All v m sid1 #q #i.CookieV(sid1,m)@i&
3.==> v=m&(Ex Data sid2 #j #k #l.
4.CookieG0(sid2,Data)@j&CookieG1(sid2,Data)@k&
5.CookieG2(sid2,Data)@l&(j<k&k<l)&sid2=sid1&
6.Data=m)&(not (Ex A B C #r.Negotiate(A,B,C)@r))"
	1.lemma **Data_Verify:**
	2."All Data m v #i #q.VerifyS(Data)@i&
	3.VP(m,v)@q==>(v=m&q<i)&(Ex Data2 #j.
	4.VerifyC(Data2)@j&Data=Data2&j<i)&
	5.not(Ex A B MK #k.SessionKey(A,B,MK)@k
	6.&k<i)"

1.lemma **DTLS_conf:** 2."not(3.Ex S C MK #i #j. 4.MasterKey(S,C,MK)@i& 5.SessionKey(S,C,MK)@j& 6.(Ex #k. 7.K(MK)@k))"	1.lemma **DTLS_I_SCYNCH:** 2"All Role_A Role_B Key PSK #i. 3. Commit(Role_A,Role_B,Key,PSK)@i==>Ex #j. 4.MasterKey(Role_A,Role_B,Key)@j& 5. not(Ex Role_A2 Role_B2 #k. 6. Commit(Role_A2,Role_B2,Key,PSK)@k 7. & not (#i=#k))
1.lemma **DTLS_traceability:** 2."All Role m #i #j. 3.Trace(Role,m) @i & Send(Role,m)@j & j<i 4.==> Ex mk. Origin(mk)@j"	

Fig. 9. Security properties of DTLS protocol.

The first stage model of the DTLS protocol is shown in Fig. 7. The rule *C_0* describes the protocol operation triggered by the client. In this process, the second line of rule *C_0* initializes the *Cookie*. The fourth line establishes a valid action fact *CookieG0* to mark the start of the cookie mechanism; rule *S_0* is the server model. *S_0* calculates the *Cookie* after receiving a message of *C_0* and establishes action fact *CookieG1* in the rule *S_0* to mark the *Cookie* calculation process. The rule *C_1* is an intermediate process in the cookie mechanism. After receiving the message of rule *S_0* that piggybacks the *Cookie*, the message of *C_1* containing the *Cookie* is sent immediately. The process is marked by action fact *CookieG2*. Finally, *S_1* receives the message from *C_1*, which determines the correctness of the *Cookie* through the *VH* function and marks the *S_1* rule with the action fact *CookieV*. Then the whole cookie verification mechanism is completed.

The second stage model of the DTLS protocol is shown in Fig. 8. After receiving *S_hint* from agent *S*, the rule *C_2* constructs pre-shared key *PSK*, message key *MK*, verification data *VDC*, and sends *VDC* to the network environment. This process consists of the fact action *MasterKey, Negotiate, Origin* and *Sd*. The action facts *MasterKey, Negotiate*, mark the verification data of key negotiation between *C* and *S*. The action facts *Origin, Sd* mark generated key *MK* and message of sending. The rule *S_2* receives the message from *C_2*, then it has the same operation as *C_2*, in addition, *S_2* needs to verify the *VDC* and send out the verification data *VDS*. This process includes the action facts *MasterKey, Negotiate, Commit, Origin, Sd* and *Trace*, where the action fact *Commit* marks that *C* and *S* have negotiated to obtain the message key, the *Trace* marks message channel between rules *C_2* and *S_2*. The *C_3* is the final rule, the verification operation of which is performed after receiving the *VDS* from *S_2* and contains action facts *SessionKey, VerifyS, Commit, and Trace*. These action facts mark the verification and key negotiation completion under rule *C_3*.

The security properties defined by the event logic and the actual model can be mapped to lemma instances that the Tamarin prover can recognize. As shown in Fig. 9, the lemma *Dos_protection* describes the implementation of cookie mechanism against DoS attacks, and the action facts *CookieG0, GookieG1*, and *GookieG2* capture the flow of cookies during the initial interaction between protocol interacting parties. The action fact *CookieV* records the currently received session ID and the cookies under the same session. The whole process can be understood as that when the cookie is correctly verified, the flow of the cookie must be carried out in strict accordance with the corresponding order, and there must be no shared key negotiated by the two parties before the cookie verification is successful. The DTLS protocol needs to ensure the integrity of the verification data, that is, the verification data cannot be tampered with by a third party. This property is described by lemma *Data_Verify*. The action fact *VerifyS, VerifyC* records the received verification data. The *VP* records a pair of verification data to be verified. The *SessionKey* is the identifier of session key *MK*. The whole process is

Table 3
Verification results of DTLS.

Mechanism	Availability	Integrity	Confidentiality	Injective synchronization	Traceability
PSK	√	√	√	×	√
Proof Step	4	4	7	5	7
RPK	√	√	√	√	√
Proof Step	3	4	2	3	3
CA	√	√	√	√	√
Proof Step	7	4	2	6	6

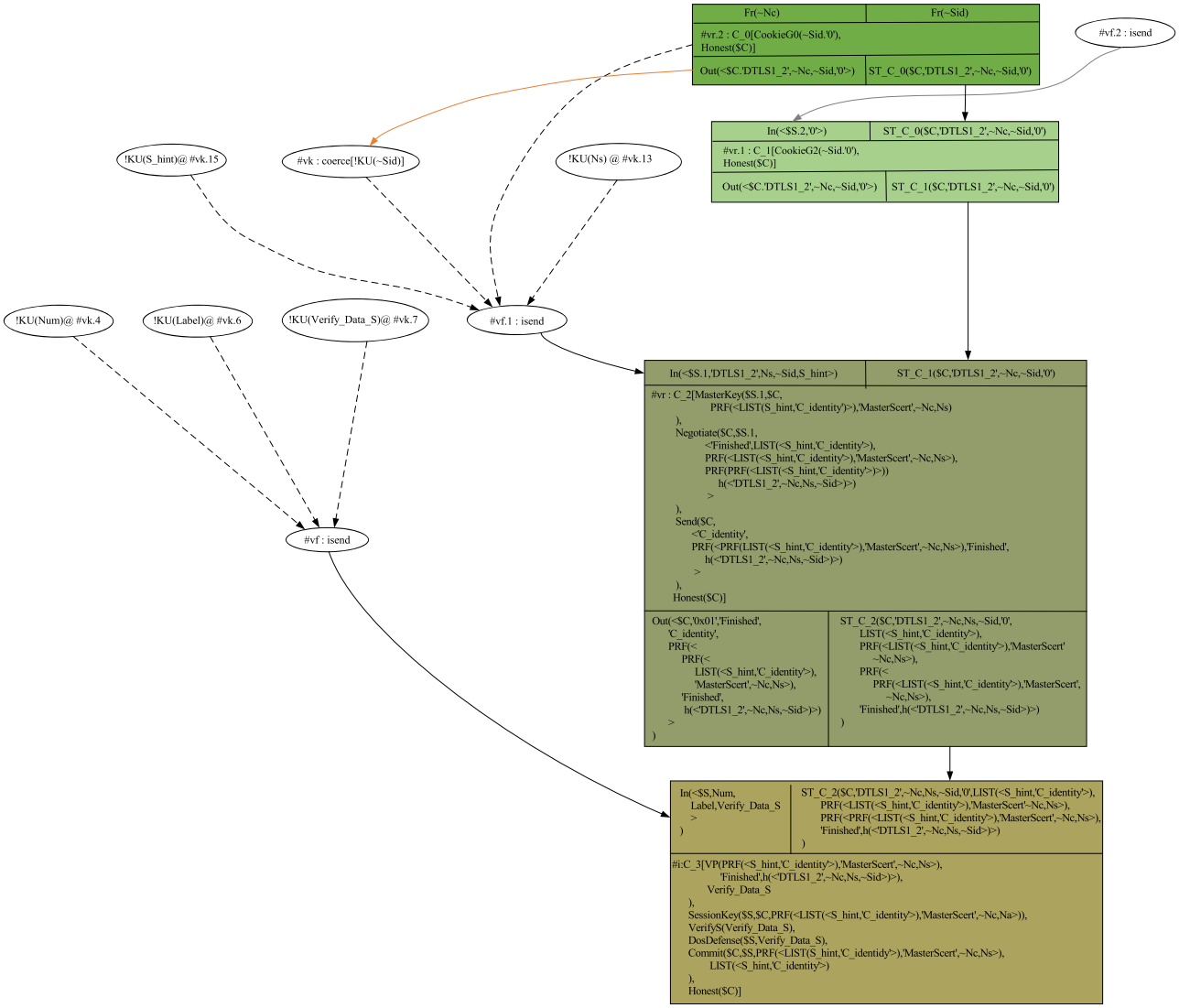

Fig. 10. Tamarin displays the attack path graph. The rectangular box refers to the action behavior in the agent state transition, the arrows represent the desired event source (in particular, the red arrow indicates that the information source is the message output during protocol execution, the dashed arrow indicates that the message source from the attacker), and the ellipse indicates the attack method.

interpreted as follows, when the protocol initiator confirms that the calculated verification data is consistent with the verification data received from the protocol responder, the protocol responder must confirm that the calculated verification data is consistent with that of the protocol initiator, and the session key *MK* cannot be determined by the protocol execution parties before the data integrity verification is successful. The confidentiality, described by lemma *DTLS_conf*, indicates that the adversary cannot obtain the session key *MK* under any circumstances. The lemma *DTLS_I_SCYHCN* describes the injective synchronization of a protocol. The action fact *Commit* marks the authentication information when the protocol agent ends execution, and the *MasterKey* records the generation of the session key *MK*. In general, injective synchronization can be comprehended as the uniqueness of the action fact *Commit*. The traceability is described by lemma *DTLS_traceability*. The DTLS specification itself does not require the protocol to meet the traceability, but the intermediate data generated in the process of key negotiation indirectly marks the message channel, so the traceability verification is only between rules *C_2, S_2*, and *C_3*.

Results

Using the Tamarin prover implement the automatic verification of

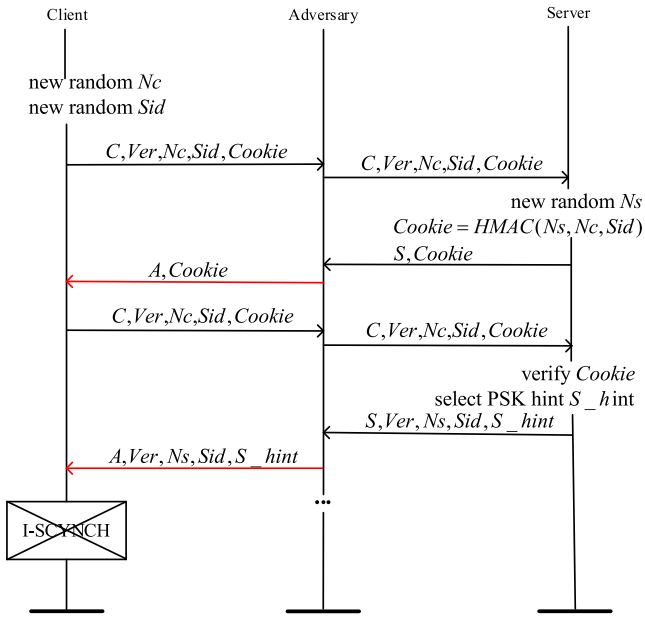

Fig. 11. The man-in-the-middle attack path.

DTLS protocol. The verification results are shown in Table 3. The DTLS protocol based on PSK does not meet injective synchronization, while the DTLS protocol based on other mechanisms meets all security properties.

As can be seen in Fig. 10, the attack occurs in the first stage. The initiator client generates a nonce *Nc* and session identifier *Sid* to initiate an initial session with the server. The adversary intercepts the message and forwards it to the server. Upon receiving the message, the server generates a nonce *Ns*, calculates cookies normally, and sends the identity and cookie to the client. The adversary intercepts message $<S,Cookie>$, modifies the message into $\langle A,Cookie\rangle$, and sends it to the client. After receiving the message, the client parses the data and thinks that the cookie is calculated by the adversary. After the session, the adversary acts as the man-in-the-middle to forward the message between the client and the server all through, making the client think that it is communicating with the adversary, however, it is interacting with the server, so this process cannot satisfy the injective synchronization. Clearly, this attack occurs because the session at the initial stage of the protocol does not contain a valid signature mechanism.

Security and performance evaluation

The verification results show that DTLS protocols with different security mechanisms exhibit varying security degrees. From the actual deployment of IoT devices, DTLS protocols based on PSK mechanism are vulnerable to man-in-the-middle attacks as shown in Fig. 11. This attack caused by the implementation of DTLS protocol in the IoT platform may be because when the user device accesses the server, the attacker truncates the message, impersonates the user to send a message to the server, and at the same time impersonates the server to respond to the user. This situation can arise in cross-network segments of the IoT system, where the attacker hides within a switch device to truncate message transmission. and there is no effective authentication mechanism between the user and the server, resulting in the inability to distinguish the attacker and the expected responder. In contrast, the DTLS protocol that uses RPK

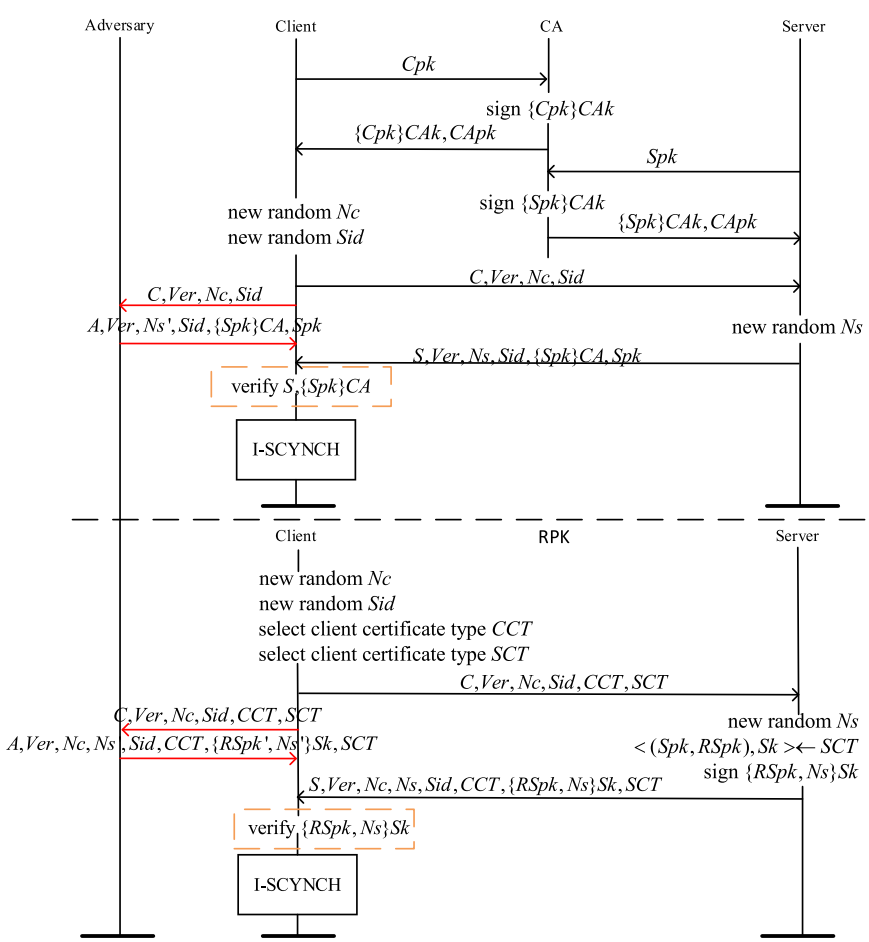

Fig. 12. Resisting Auth. type attack.

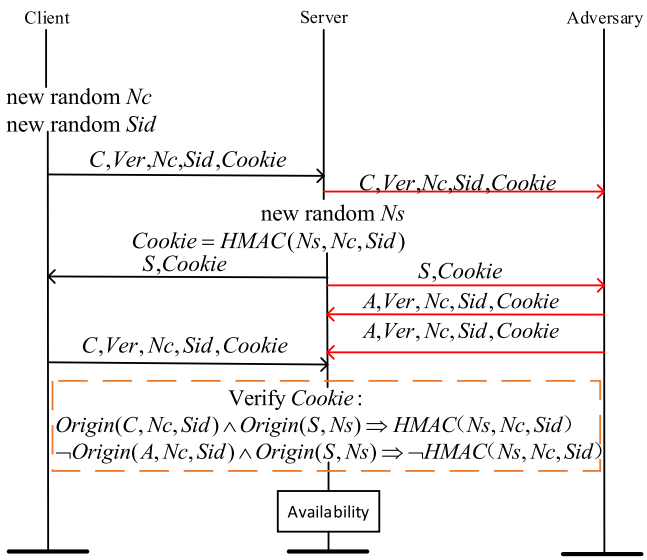

Fig. 13. Resisting DoS attack.

and CA can effectively defend against man-in-the-middle attacks, impersonation attacks, sybil attacks, and other attacks targeting protocol authentication. This is due to the protocol contains the signature information in the first phase of the message as shown in Fig. 12. Through effective signature and authentication means, DTLS protocol can avoid the attack of protocol authentication.

DTLS guarantees availability against DoS attacks. Assume that a malicious third party attacks the DTLS protocol. As shown in Fig. 13, the attacker launches a DoS attack on the first stage of the protocol. At the beginning of the protocol, the adversary obtains the handshake request command broadcast by the user, constructs the Dos attack instruction after the server broadcasts the cookie to the user, and then sends the attack instruction again to the server. The server receives the message from the user and the adversary, then needs to verify the cookie value in the command. The next session can be conducted only after the cookie authentication is passed. The cookie verified by the server is generated by the nonce *Nc* generated by the user and the *Ns* generated by the server itself. Therefore, the adversary cannot pass the authentication, and the session between the adversary and the service cannot be carried out.

The DTLS protocol implements integrity to defend against attacks such as data tampering and false data injection. The final attack target of the adversary is to tamper with the application data. Because the encrypted application data in the DY model can only be deconstructed through the corresponding session key, the implementation of the corresponding attack is in the second stage of the protocol, as shown in Fig. 14. In the second stage of the protocol, the adversary constructs the message through the data broadcast in the first stage, but because the adversary cannot generate the pre-shared key consistent with the user in the process of protocol interaction, it cannot pass the data integrity verification, then it cannot get the session key and deconstruct the encrypted application data.

Choose a security mechanism with stronger performance to achieve stronger security, but complex security mechanisms will occupy more hardware and network resources, and reduce the flexibility of IoT device deployment. For many IoT device requirements, it is not necessary to obtain the highest degree of security, it is only necessary to select the right security mechanism for the specific implementation. As the IoT version of the TLS protocol, DTLS reduces the reliability of data transmission (UDP transmission), reduces the number of handshakes, and cuts the TLS function to meet the requirements of the IoT. For the DTLS protocol, PSK can ensure the lowest security but requires the least resources, which is suitable for occasions with low security requirements. RPK has a stronger security guarantee than PSK but consumes more resources, and CA has the strongest security guarantee, which is suitable for environments with high security requirements but insensitive to data transmission delay. The performance evaluation of DTLS protocol verification is shown in

Table 4. The verification program runs in Ubuntu16 system with intel i7800k(4-core) processor and 4 G memory.

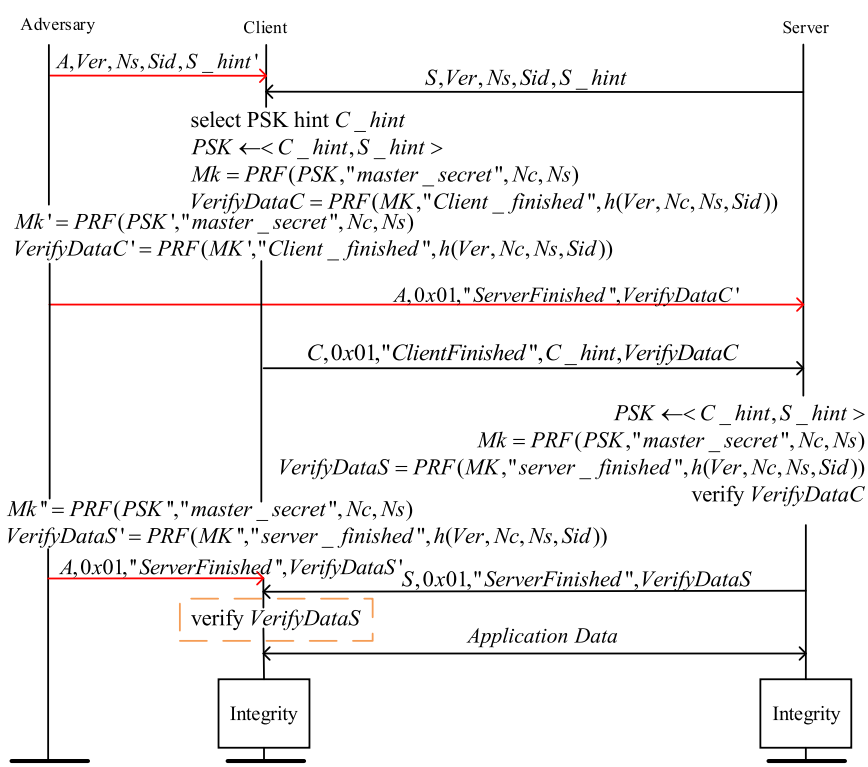

Fig. 14. Resisting integrity type attack.

Table 4
Performance evaluation of verification.

Mechanism	Security degree	% CPU	% MEM	Maximum steps	Original maximum steps	Time
PSK	low	24.3	2.18	7	21	0.39s
RPK	medium	8	2.17	4	10	3.12s
CA	high	19.7	2.19	7	16	1.27s

Discussion

The above work shows that our proposed symbolic model can be applied to the analysis of secure IoT protocol specification. Moreover, by extracting the finite state machine of the IoT system, the model is able to analyze the implementation of the actual IoT system, as shown in Fig. 15. First of all, for the actual deployment of the IoT and standardized documents, it can abstract the interaction process of the device. In this step, tool Wireshark can be used to parse device interaction, extract device working behaviors in different roles, and then refine them according to the standardized documents generated during device and system development. The goal of step 2 is to extract the protocol state machine and generate rules as the input model for Tamarin. In this process, the basic model is generated by model definition module of LoET, and then the model structure is optimized by combining the security property module of LoET and the requirements of protocol security in the specification document. The security requirements of the actual IoT system are determined before the development and deployment of the system, so step 3 to generate the actual IoT system security properties that Tamarin can recognize, it is necessary to instantiate the security property module of LoET according to the actual needs through the specification document and configuration document, and the security property module after instantiation is used as the formal verification target. Step 4 further optimize the Tamarin model according to security objectives and actual verification needs. The axiom module of LoET can be used to specify the characteristics of the verified IoT system, the method module can be used to add abstract events to simplify the model, or add inference rules to assist proof, and use MB to improve the verification efficiency. This process generates the final model to be verified, and researchers can add or subtract modules according to specific needs, making verification more flexible. Step 5 is the formal verification of the

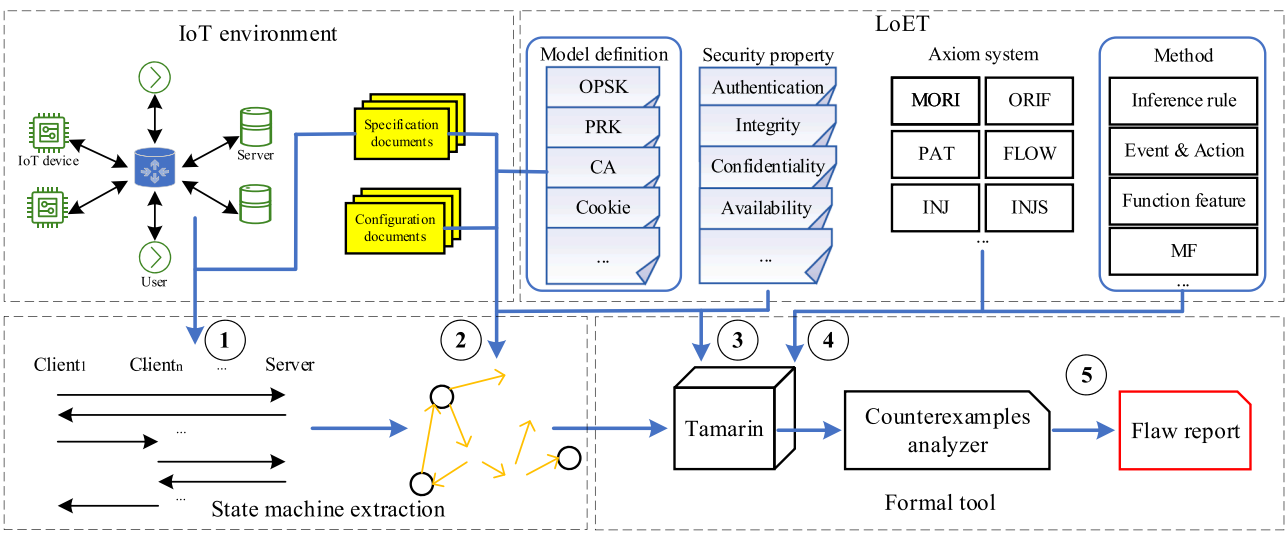

Fig. 15. Practical implementation of the model in real-world IoT systems.

Table 5
Performance comparison of symbolic model. • refer to common security mechanisms such as symmetric encryption, asymmetric encryption, and signature verification. - indicates no this part or no find defect. Supported security properties refer to the ability to analyze these security properties, supported security mechanisms refer to the ability to model the mechanism, proof steps refer to the steps to prove security properties, and supported tools refer to tools that can automate verification. * Indicates that a protocol defect is found.

Articles	Protocols	Supported security properties	Supported mechanism	Proof steps	Results	Supported tools
This work	DTLS, CoAP, SRTP, PUF	Availability, Integrity, Confidentiality, I-SCYNCH, Traceability	PSK, RPK, CA, cookie, •	4–7	*	Tamarin
Song et al., 2022	PUF	Authentication	•	10	–	–
Nakabayashi and Yuki, 2021	DH, NS, HMQV, NAXOS, CMQV	Confidentiality	•	4–7	*	Tamarin
Yang et al., 2016	Group Protocol	Authentication	PSK, CA, •	4, 5	–	Scyther
Vattakunnel et al., 2016	CoAP	Availability	–	4	–	SPIN
Kim et al., 2017	CoAP, MQTT, Sigfox, LoRa	Availability, Confidentiality	PSK, •	4–6	–	Tamarin
Chen et al., 2022	CoAP	Availability, Integrity	RPK, CA, •	9	*	PAT
Wang et al., 2022	MQTT, CoAP, AMQP	Availability, Authentication, Confidentiality	•	5	*	Tamarin
Akhtar and Zahoor, 2021	MQTT	Availability	•	7	–	TLA+
Wang et al., 2021	SKKE	Confidentiality, Authentication	•	7	*	–
Cremers et al., 2017	TLS1.3	Confidentiality, Authentication	PSK, CA, •	7	*	Tamarin
Basin et al., 2018	5G	Confidentiality, Authentication	•	8	*	Tamarin
Li et al., 2020	ZigBee1.0/3.0	Availability, Authentication, Confidentiality	•	6	*	Tamarin
Tran et al., 2023	TLS1.0	Confidentiality, Authentication	•	10	–	CiMPG

model to find the path of violating the security properties in the model, that is, the counterexample of the security property specification. By analyzing counterexamples, we can get the specific attack behavior and generate the final error report.

The symbolic model of IoT protocol security analysis proposed can be effectively applied to the security analysis of protocol specifications and actual IoT systems, and its simple and convenient use makes it easy to be deployed in actual IoT systems. However, this method still has some limitations. When considering IoT systems that include a large number of devices, or with a large number of users, there are certain limitations in application efficiency. The optimization of model efficiency focuses on the MB method proposed in step 4, which can greatly improve the verification efficiency. However, the implementation of state machine extraction in step 1 and step 2 needs to be completed manually, which leads to the cumbersome process of model extraction. When there are hundreds of IoT devices, manual extraction of the model will become difficult and unreliable. The labor and time costs involved will be unbearable. Therefore, before using this symbolic model, it is necessary to evaluate the scale of the IoT system, considering the complexity of the dynamic interaction of the IoT protocol, and generally speaking, in order to ensure that the effective verification of the IoT system, the cost of steps 1 and 2 should be as close as possible to the cost of steps 3 to 5.

Considering applicability, functionality and verification efficiency, our proposed symbolic model is significantly superior to the method mentioned in 'Related Work' in terms of performance. As shown in Table 5, our proposed method can simultaneously establish models of multiple security mechanisms and analyze various typical IoT security properties, with the largest number of similar methods. Meanwhile, our method has fewer proof steps, is greatly superior to most methods in proof efficiency, and supports the analysis and verification of multiple automated tools, with strong flexibility.

Conclusion and future work

This paper proposes a symbolic model to analyze secure IoT protocols based on event logic. The basic IoT architecture, message equivalence, form equivalence, pattern, share, injectivity, and verification match are defined precisely. The first receiving predicate *Firstrcv* and origin predicate *Origin* are proposed to solve the analysis defect of the original event logic, and a series of related inference rules are given. On the basis of these new extensions, we give the formal definitions of event logic for injective synchronization, confidentiality, integrity, availability, and traceability. For the analysis of secure IoT protocols, we propose formal definitions for special mechanisms of IoT protocols such as PSK, cookie, RPK, and honest CA, and give corresponding theorems and lemmas, as well as establish a proof system for IoT protocol mechanisms. Considering the problem of redundant proof in event logic, we propose the MB reduction method, which greatly reduces the proof process. Using the symbolic model proposed in this paper, we analyze the DTLS, CoAP, SRTP, and PUF protocols, and find that some protocols fails to achieve specified security properties.

The proof results show that the symbolic model proposed in this paper can be used in the security proof of IoT protocol, but the method still has room for improvement. With the continuous development of advanced encryption technologies and the increased demand for high security in IoT devices, technologies such as post-quantum cryptography (PQC) and homomorphic encryption (HE) have gradually begun to be applied to IoT devices. However, existing symbolic models lack analytical support for these advanced encryption schemes.

Traditional public-key cryptography algorithms may be cracked by quantum algorithms, which makes the current cryptographic primitives unable to model quantum cryptography effectively, and the axiom and proof system under traditional cryptography will no longer be applicable. Homomorphic encryption allows computations to be performed directly on the ciphertext, enabling secure data processing without decryption, which creates additional challenges for protocol analysis because of the need to consider stronger threat models over existing methods and build completely different axiom systems. Future work should focus on extending the symbolic model to integrate post-quantum cryptographic primitives and homomorphic cryptographic schemes to ensure that security analysis can adapt to emerging cryptographic environments. At the application level, IoT devices often need to connect to multiple platforms or support multi-user access, so the security of group communication protocols must be ensured. Dynamic group communication (such as one-to-many, many-to-many communication) security needs special attention. In addition, IoT environments often involve multiple protocols co-existing, or even different versions of protocols. Such complexity requires that the symbolic model has the ability to analyze and verify the security of the composition protocol. In contrast to single protocol analysis, composition protocol analysis needs to evaluate the interactions between multiple protocols and determine whether their composition will introduce unexpected security vulnerabilities.

In summary, future work will focus on the following two key aspects: (1) Extend the symbolic model to support post-quantum encryption and homomorphic encryption, and ensure its compatibility with emerging encryption technologies; (2) Enhance the security analysis capability of the model for dynamic group communication and complex composition protocols, so as to meet the actual security challenges of the IoT.

CRediT authorship contribution statement

Zehuan Li: Writing – review & editing, Writing – original draft, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Meihua Xiao:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Funding acquisition, Data curation, Conceptualization. **Yangping Xu:** Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis, Data curation. **Fangping Chen:** Writing – original draft, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Huaibin Shao:** Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis. **Sufen Yan:** Writing – original draft, Visualization, Software, Investigation, Formal analysis, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was funded by the National Natural Science Foundation of China, grant number 62362033, 61962020), Double Thousand Talent Plan of Jiangxi Province (No. jxsq2023201009), Natural Science Foundation of Jiangxi Province (No. 20224ACB202006).

Appendix A

By proving the match of basic sequences, we deduce that the protocol model can satisfy the security properties formula, as shown below. Fig 16

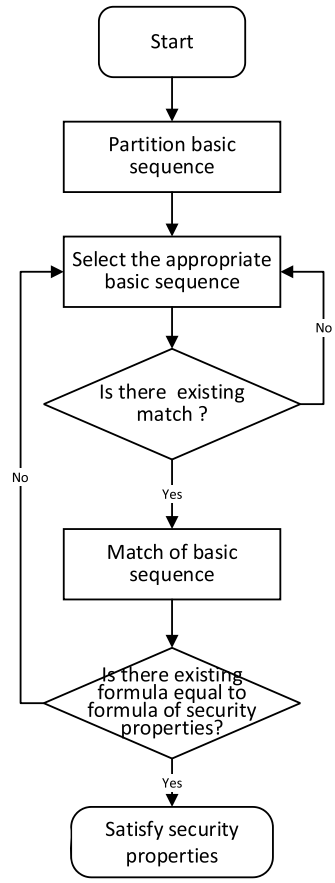

Fig. 16. Proof process of event logic.

Appendix B

Table 6

Table 6
Verification results of other protocols.

Protocol	Availability	Integrity	Confidentiality	Injective synchronization	Traceability	Time
CoAP	√	√	√	√	√	2.81s
Proof step	3	4	2	3	3	
SRTP	×	√	√	×	√	6.54s
Proof step	8	3	4	5	8	
PUF	×	√	√	√	√	0.81s
Proof step	7	15	27	8	9	

Proof summary of CoAP protocol

We analyzed and proved the CoAP protocol (Shelby et al., 2014). The interaction of the CoAP protocol is as follows:

$$A \rightarrow B : A, Ver, Na, Sid$$
$$B \rightarrow A : B, Ver, Na, Nb, Sid, \{|Pk_A, N_b|\}_{k_B}$$
$$A \rightarrow B : A, \{|Pk_A, N_a|\}_{k_A}, PRF\left(g^{k_A * Pk_B}, N_a, N_b\right)$$
$$B \rightarrow A : B, PRF\left(g^{k_B * Pk_A}, N_a, N_b\right)$$

To get eventualization of CoAP protocol, the basic sequence is as follows:

$I_1 = New(Na), New(Sid), Send(A, Ver, Na, Sid)$
$I_2 = I_1, Rcv(B, Ver, Na, Nb', Sid, S_1'),$
$Verify(\{|Pk_B, Nb'|\}k_B', S_1')$
$I_3 = I_2, Sig(S_2, Pk_A, Na, k_A), PRF(Mk, k_A, Pk_B),$
$PRF(S_3, Mk, k_A, Na, Nb'), Send(A, S_2, Pk_A, S_3)$
$I_4 = I_3, Rcv(B, S_5'), Verify(S_3, S_5')$

$R_1 = Rcv(A, Ver, Na', Sid'),$
$New(Nb), Sig(S_4, Pk_B, N_b, k_B),$
$Send(B, Ver, Na', Nb, Sid', S_4)$
$R_2 = R_1, Rcv(A, S_2', Pk_A, S_3'),$
$Verify(\{Pk_b, Nb\}k_B, S_2')$
$R_3 = R_2, PRF(Mk, k_B, Pk_A),$
$PRF(S_5, Mk, k_B, Na', Nb),$
$Verify(S_5, S_3'), Send(B, S_5)$

Confidentiality:

1) By employing the predicate *Firstrcv* and *Origin*, search the data sources in each base sequence.
2) Prove the direction of the message flow in 1) by Theorem 1, Lemma 2.
3) Suppose that a message can be acquired by the adversary. According to MORI, ORIF, and OWF, it can be obtained that the corrupted message is inability to deduce any secret data, and the protocol agent cannot release any secret message.
4) The confidentiality can be proved.

Integrity:

1) Determine the origin of the encrypted data key by predicate *Origin*.
2) Prove the existence of matching basic sequences by Definition 6, axioms VERO, VERM.
3) Use axiom PAT, OWF, and Lemma 2 to prove that messages cannot be tampered with.
4) Complete proof of the integrity of the CoAP protocol.

Availability:

1) Use function *P* to extract the form of event *Send* and *Rcv* and get the protocol message structure.
2) Use axioms MORI, ORIF, PAT, and FLOW. It is proved that the *Send* event and *Rcv* event message structures are equivalent.
3) The axioms INJ, and INJS are used to prove that the match events *Send* and *Rcv* in 2) have a sequential relation, with *Send*<*Rcv*.
4) This proves the availability of the CoAP protocol.

Traceability:

1) There is no security requirement for traceability in the CoAP protocol standards document, but the protocol can still guarantee traceability through the session ID and generated nonces.
2) By employing the predicate *Firstrcv* and *Origin*, search the data sources in each base sequence.
3) Prove the direction of the message flow in 1) by ESID, FLOW.
4) Finally, the INJS axioms can be used to prove that each pair of *Send* and *Rcv* events are unique, that is, to prove that the CoAP protocol is traceability.

Proof summary of SRTP (Secure Real-Time Transport Protocol)
We analyzed and proved the SRTP protocol (Xiao et al., 2024). The interaction of the SRTP protocol is as follows:

$A \rightarrow B : A, Sid, Na, pkA, Ver$
$B \rightarrow A : B, Sid, Nb, pkB, \{|pkA|\}_{k_B}, Ver$
$A \rightarrow B : A, Sid, \{|HMAC(Sid, Na, Nb, Ver)|\}_{Pk_A}$
$B \rightarrow A : B, Na, Nb, \{|HMAC(HMAC(Sid, Na, Nb, Ver))|\}_{pk_B}$
$A \rightarrow B : B, Na, Nb, PRF(g^{k_A * pk_B}, HMAC(HMAC(Sid, Na, Nb, Ver)))$
$B \rightarrow A : B, PRF(g^{k_B * pk_A}, HMAC(HMAC(Sid, Na, Nb, Ver)))$

To get eventualization of SRTP protocol, the basic sequence is as follows:

$I_1 = New(Sid), New(Na),$
$Send(A, Sid, Na, pk_A, Ver)$
$I_2 = I_1, Rcv(B, Sid, Nb', pkB, S_1', Ver),$
$Verify\left(\{pk_A\}_{pk_B}, S_1'\right)$
$I_3 = I_2, HMAC(S_2, Sid, Na, Nb, Ver),$
$Enc(S_3, S_2, pk_B), Send(A, Sid, S_3)$
$I_4 = I_3, Rcv(B, Na, Nb', S_4'), HMAC(S_5, S_2)$
$I_5 = I_4, PRF(S_6, g^{k_A * pk_B}, S_5), Verify(S_6, S_4')$

$R_1 = Rcv(A, Sid', Na', pk_A, Ver),$
$New(Nb), Sig(S_1, pkA, k_B)$
$R_2 = R_1, Send(B, Sid', Nb, pk_B, S_1, Ver)$
$R_3 = R_2, Rcv(A, Sid', S_3'),$
$Dec(S_2', k_B, S_3'), HMAC(S_4, S_2')$
$R_4 = R_3, PRF(S_5, g^{k_B * pk_A}, S_4),$
$Send(B, Na', Nb, S_5)$

Confidentiality:

1) Starting with the last basic sequence of the responder, the corresponding basic sequence can be obtained according to AxiomR and AxiomD.
2) By axioms VERM and Theorem 1, we can get the match and data flow of the basic sequence in which the signature exists.
3) Use axioms MORI and FLOW for basic sequences without signatures to get the match and data flow of the basic sequences.
4) Suppose that a message can be acquired by the adversary. According to MORI, ORIF, and OWF, it can be obtained that the corrupted message is inability to deduce any secret data, and the protocol agent cannot release any secret message, exactly follow the established channel transmission
5) Therefore, it is proved that the obtained SRTP satisfies the confidentiality.

Integrity:

1) Starting with the last basic sequence of the responder, through the axioms AxiomR and VERM, it is proved that there is a match for the signature session.
2) According to the equivalence of messages in match of *Send* and *Rcv* and axiom PAT, form equivalence and message equivalence can be obtained, then SRTP protocol satisfies integrity under this message channel.
3) For a basic sequence that contains only the events *Enc* and *Dec*, the match of the basic sequence is proved by AxiomD, MORI, and ORIF. Finally, PAT is used to prove that the message channel satisfies integrity.
4) It is proved that all message channels of SRTP protocol meet the integrity, so the whole protocol meets the integrity.

Availability:

1) As with the proof of the security properties above, the match of all basic sequences is proved.
2) On the basis of match proof, we can get the form equivalence of message through PAT axiom.
3) Through AxiomR and VERM, we can get the partial order relation of events *Send* and *Rcv* in the basic sequence of existing signed sessions, that is, $Send < Rcv$.
4) However, for other basic sequences, the partial order relation $Send < Rcv$ cannot be proved.

Proof summary of PUF (Physical Unclonable Functions) protocol

We analyzed and proved the PUF protocol (Song et al., 2022). The interaction of the PUF protocol is as follows:

$A \rightarrow B : A, Sid, Ver$
$B \rightarrow A : B, Sid, Ver, n_1, \{|k'|\}_k$
$A \rightarrow B : A, Sid, Ver, \{|n_1, n_2|\}_{k'}$
$B \rightarrow A : B, Sid, Ver, \{|n_3, n_2|\}_{k'}$

To get eventualization of PUF protocol, the basic sequence is as follows:

$I_1 = New(Sid), Send(A, Sid, Ver)$
$I_2 = I_1, Rcv(B, Sid, Ver, n_1', S_1')$
$I_3 = I_2, Dec(k'', k, s_1'), New(n_2),$
$Enc(S_2, n_1' \parallel n_2, k''), Send(A, Sid, Ver, S_2)$
$I_4 = I_3, Rcv(B, Sid, Ver, S_3')$
$I_5 = I_4, Dec(n_3' \parallel n_2, K'', S_3')$

$R_1 = Rcv(A, Sid', Ver)$
$R_2 = R_1, New(n_1),$
$Enc(S_1, k', k), Send(B, Sid', Ver, n_1, S_1)$
$R_3 = R_2, Rcv(A, Sid', Ver, S_2')$
$R_4 = R_3, Dec(n_2' \parallel n_1, k', S_2'),$
$New(n_3), Enc(n_3 \parallel n_2', k', s_3), Send(B, Sid', Ver, S_3)$

Confidentiality:

1) Start with the initiator's last basic sequence to prove that there is a match of each basic sequence.
2) According to axioms AxiomR and AxiomD, the match relation of each basic sequence can be proved.
3) Suppose that a message can be acquired by the adversary.
4) Get the messages flow in the basic sequence according to MORI, ORIF, and FLOW, then based on the basic definition of events, it can be obtained that the corrupted message is inability to deduce any secret data, and the protocol agent cannot release any secret message.

Integrity:

1) It is proved on the basis of basic sequence match.
2) Through the axiom PAT we can get a matching basic sequence in which the messages in *Send* and *Rcv* are form equivalence.
3) Then through the axioms FLOW, AxiomR and AxiomD, we can obtain the data of form equivalent in 2) and also satisfy the message equivalence.
4) Through the steps 2) and 3), it can be proved that the data in the channels existing in all the matched basic sequences meet the form equivalence and message equivalence, then it is proved that the PUF has integrity.

Availability:

1) Proof based on integrity and confidentiality.
2) According to the axioms AxiomD and FLOW, there is a partial order relation between the events *Send* and *Rcv*, with *Send*<*Rcv* in basic sequence I_2-I_5, R_2-R_4.
3) Then, I_1 and R_1 lack basic condition to proof partial order relation.

Data availability

All relevant data are within the manuscript.

References

Akhtar, S., Zahoor, E., 2021. Formal specification and verification of MQTT protocol in PlusCal-2. Wirel. Pers. Commun. 119, 1589–1606. https://doi.org/10.1007/s11277-021-08296-4.

Asghari, P., Rahmani, A.M., Javadi, H.H.S., 2018. Service composition approaches in IoT: a systematic review. J. Netw. Comput. Appl. 120, 61–77. https://doi.org/10.1016/j.jnca.2018.07.013.

Basin, D.A., Dreier, J., Hirschi, L., Radomirovic, S., Sasse, R., Stettler, V., 2018. In: A Formal Analysis of 5G Authentication. Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. Toronto Canada. ACM, pp. 1383–1396. https://doi.org/10.1145/3243734.3243846.

Bickford, M., Constable, R.L., 2010. Automated proof of authentication protocols in a logic of events. In: International Joint Conference on Automated Reasoning (IJCAR) Verification Workshop. Edinburgh, UK, pp. 20–21. https://doi.org/10.29007/r7n1.

Blanchet, B., Vincent, C., Véronique, C., 2022. Proverif with lemmas, induction, fast subsumption, and much more. In: 2022 IEEE Symposium on Security and Privacy (SP). IEEE. https://doi.org/10.1109/sp46214.2022.9833653.

Blanchet, B., 2012. Security protocol verification: symbolic and computational models. In: Degano, P., Guttman, J.D. (Eds.), Principles of Security and Trust. POST 2012. Lecture Notes in Computer Science. Springer, Berlin, Heidelberg, pp. 3–29. https://doi.org/10.1007/978-3-642-28641-4_2.

Cervesato, I., 2021. The Dolev-Yao intruder is the most powerful attacker. In: 16th Annual Symposium on Logic in Computer Science—LICS. 2010(1).

Chen, S., Li, R., Zhu, H., et al., 2022. Formalization and verification of group communication CoAP using CSP. In: Shen, H., et al. (Eds.), Parallel and Distributed Computing, Applications and Technologies. PDCAT 2021. Lecture Notes in Computer Science. Guangzhou. China. Springer, pp. 616–618. https://doi.org/10.1007/978-3-030-96772-7_58.

Cremers, C.J., Horvat, M., Hoyland, J.S., Scott, S., Merwe, T.V, 2017. In: A Comprehensive Symbolic Analysis of TLS 1.3. Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. Dallas, USA. ACM, pp. 1773–1788. https://doi.org/10.1145/3133956.3134063.

Fremantle, P., Scott, P., 2017. A survey of secure middleware for the Internet of Things. PeerJ Comput. Sci. 3, e114. https://peerj.com/articles/cs-114/.

Hasan, M.K., Shafiq, M., Islam, S., Pandey, B., El-Ebiary, Y.A., Nafi, N.S., Rodriguez, C.R., Vargas, D.E, 2021. Lightweight cryptographic algorithms for guessing attack protection in complex internet of things applications. Complex 2021, 1–13. https://doi.org/10.1155/2021/5540296.

Hcine, J., Hafaiedh, I.B, 2018. Formal-Based Modeling and Analysis of a Network Communication Protocol for IoT: MQTT Protocol. Smart Innovation, Systems and Technologies. Springer, Genoa, Italy, pp. 350–360. https://doi.org/10.1007/978-3-030-21009-0_34.

Hofer-Schmitz, K., Stojanović, B., 2020. Towards formal verification of IoT protocols: a review. Comput. Netw. 174, 107233. https://doi.org/10.1016/j.comnet.2020.107233.

Kakkar, L., Gupta, D., Saxena, S., Tanwar, S., 2021. IoT Architectures and its security: a review. In: proceedings of the Second International Conference on Information Management and Machine Intelligence. Singapore, 166. Springer, pp. 110–117. https://doi.org/10.1007/978-981-15-9689-6_10.

Karam, S.N., Bilal, K., Khan, A.N., Shuja, J., Abdulkadir, S.J., 2024. Energy-efficient routing protocol for reliable low-latency internet of Things in oil and gas pipeline monitoring. PeerJ. Comput. Sci. 10, e1908. https://doi.org/10.7717/peerj-cs.1908.

Kim, J.Y., Holz, R., Hu, W., Jha, S., 2017. Automated analysis of secure Internet of Things protocols. In: Proceedings of the 33rd Annual Computer Security Applications Conference. FL, USA. ACM, pp. 238–249. https://doi.org/10.1145/3134600.3134624.

Li, Y., Schäge, S., Yang, Z., Kohlar, F., Schwenk, J., 2014. On the security of the pre-shared key ciphersuites of TLS. In: Krawczyk, H. (Ed.), Public-Key Cryptography – PKC 2014. PKC 2014. Lecture Notes in Computer Science. Springer, Berlin, Heidelberg, pp. 669–684. https://doi.org/10.1007/978-3-642-54631-0_38.

Li, L., Podder, P., Hoque, M.E., 2020. A formal security analysis of ZigBee (1.0 and 3.0). In: Proceedings of the 7th Symposium on Hot Topics in the Science of Security. Lawrence, USA. ACM, pp. 1–11. https://doi.org/10.1145/3384217.3385617.

Liu, F., Yan, Y., Sun, Y., Liu, J., Li, D., Guan, Z., 2022. Extremely lightweight PUF-based batch authentication protocol for end-edge-cloud hierarchical smart grid. Secur. Commun. Netw. 2022, 1–14. https://doi.org/10.1155/2022/9774853.

Luo, S., Hu, T., Han, N., Qian, Y., 2024. A smart-contract-based policy-domain access control framework for distributed industrial IoT. IEEe Internet. Things. J. 11, 11427–11443. https://doi.org/10.1109/JIOT.2023.3329541.

Modiri, M.M., Salmasizadeh, M., Mohajeri, J., Khalaj, B., 2023. Two protocols for improving security during the authentication and key agreement procedure in the 3GPP networks. Comput. Commun. 211, 286–301. https://doi.org/10.2139/ssrn.4402924.

Nakabayashi, M., Yuki, O., 2021. Verification method of key-exchange protocols with a small amount of input using Tamarin Prover. In: Proceedings of the 2021

International Symposium on Advanced Security on Software and Systems. https://doi.org/10.1145/3457340.3458301.
Peng, Y., Zhao, S., Wang, H., 2024. IoT-aimed health monitoring scheme for power semiconductor devices based on Mahalanobis processing. IEEe Internet. Things. J. 11, 11246–11258. https://doi.org/10.1109/JIOT.2023.3330799.
Ping, L., Shengjun, L., Hongyang, Y., Lishan, K., Teng, H., Alzubair, H., 2020. A group identification protocol with leakage resilience of secret sharing scheme. Complexity. 2020, 1–13. https://doi.org/10.1155/2020/1784276.
Pokorni, S.J, 2019. Reliability and availability of the internet of things. Military Technical Courier 67 (3), 588–600. https://www.redalyc.org/pdf/6617/661770393007.pdf.
Rescorla, E., Modadugu, N., 2012. Datagram transport layer security version 1.2. RFC 6347, 1–32. https://doi.org/10.17487/RFC6347.
Riyadi, E., Putra, A.E., Priyambodo, T.K., 2021. Improvement of nuclear facilities DNP3 protocol data transmission security using super encryption BRC4 in SCADA systems. PeerJ. Comput. Sci. 7, e727. https://doi.org/10.7717/peerj-cs.727.
Shelby Z., Hartke K., Bormann C. 2014. The constrained application protocol (CoAP). https://www.rfc-editor.org/rfc/rfc7252.
Song, J., Xiao, M., Zhang, T., 2022. Proving authentication property of PUF-based mutual authentication protocol based on logic of events. Soft. Comput. 26, 841–852. https://doi.org/10.1007/s00500-021-06163-9.
Tran, D.D., Mon, T.W., Ogata, K., 2023. Transport layer security 1.0 handshake protocol formal verification case study: how to use a proof script generator for existing large proof scores. PeerJ Comput. Sci. 9, e1284. https://doi.org/10.7717/peerj-cs.1284.
Vattakunnel, A.J., Kumar, N.S., Kumar, G.S, 2016. Modelling and verification of CoAP over routing layer using SPIN model checker. Procedia Comput. Sci. 93, 299–308. https://doi.org/10.1016/J.PROCS.
Wang X., Gu C., Wei F., Lu S. 2021. Security and privacy for edge-assisted internet of things Security proof for the SKKE protocol. Secur. Commun. Netw. 2021, 1–14. https://doi.org/10.1155/2021/9029664.
Wang, Q., Ji, S., Tian, Y., Zhang, X., Zhao, B., Kan, Y., Lin, Z., Lin, C., Deng, S., Liu, A.X., Beyah, R.A, 2022. MPInspector: a systematic and automatic approach for evaluating the security of IoT messaging protocols. In: USENIX Security Symposium. Vancouver, Canada. USENIX. https://doi.org/10.48550/arXiv.2208.08751.
Wouters, P., Tschofenig, H., Gilmore, J., Weiler, S., Kivinen, T., 2014. Using raw public keys in transport layer security (TLS) and datagram transport layer security (DTLS). RFC 7250, 1–18. https://doi.org/10.17487/RFC7250.
Xiao, M., Bickford, M., 2009. Logic of events for proving security properties of protocols. In: 2009 International Conference on Web Information Systems and Mining. Shanghai, China. IEEE, pp. 519–523. https://doi.org/10.1109/WISM.2009.111.
Xiao, M., Zhao, H., Yang, K., 2022. A formal analysis method for composition protocol based on model checking. Sci. Rep. 12 (8493). https://doi.org/10.1038/s41598-022-12448-2.
Xiao, M., Chen, Q., Li, Z., Chen, Y., Xu, R., 2023. Formal security analysis of ECC-based RFID in logic of events theory. Electronics. (Basel) 12 (15), 3286. https://doi.org/10.3390/electronics12153286.
Xiao, M., Zhong, Y., Li, Z., Chen, F., 2024. Formal analysis of DTLS-SRTP combined protocol based on logic of events. Appl. Sci. 14 (5), 1804. https://doi.org/10.3390/app14051804.
Xiong, J., Ren, J., Chen, L., Yao, Z., Lin, M., Wu, D., Niu, B., 2018. Enhancing privacy and availability for data clustering in intelligent electrical service of IoT. IEEe Internet. Things. J. 6 (2), 1530–1540. https://doi.org/10.1109/JIOT.2018.2842773.
Yang, H., Vladimir, A.O., Andreas, P., 2016. Verifying group authentication protocols by Scyther. J. Wirel. Mob. Netw. Ubiquitous Comput. Dependable Appl. 7 (2), 3–19.
Yogesh, P.R, 2020. Formal verification of secure evidence collection protocol using BAN logic and AVISPA. Procedia Comput. Sci. 167 (2020), 1334–1344. https://doi.org/10.1016/j.procs.2020.03.449.
You, I., Yoshiaki, H., Kouichi, S., 2011. Enhancing SVO logic for mobile IPv6 security protocols. J. Wirel. Mob. Netw. Ubiquitous Comput. Dependable Appl. 2 (3), 26–52. https://isyou.info/jowua/papers/jowua-v2n3-2.pdf.
Yu, L., Zhi-Yao, Y., Ze-Peng, Z., 2021. Extension of PCL theory and its application in improved CCITT X. 509 analysis. Int. J. Netw. Secur. 23 (2), 305–313. http://ijns.jalaxy.com.tw/contents/ijns-v23-n2/ijns-2021-v23-n2-p305-313.pdf.
Zanella, A., Bui, N., Castellani, A., Vangelista, L., Zorzi, M., 2014. Internet of things for smart cities. IEEe Internet. Things. J. 1 (1), 22–32. https://doi.org/10.1109/JIOT.2014.2306328.
Zhong, X., Xiao, M., Zhang, T., Yang, K., Luo, Y., 2022. Proving mutual authentication property of RCIA protocol in RFID based on logic of events. Chin. J. Electronics 31 (1), 79–88. https://doi.org/10.1049/cje.2021.00.101.